Artificial Intelligence Review (2025) 58:138
https://doi.org/10.1007/s10462-025-11134-9

An actor-critic based recommender system with context-aware user modeling

Maryam Bukhari[1] · **Muazzam Maqsood**[1] · **Farhan Adil**[1,2]

Accepted: 6 February 2025 / Published online: 19 February 2025
© The Author(s) 2025

Abstract
Recommendation systems empower users with tailored service assistance by learning about their interactions with systems and recommending items based on their preferences and interests. Typical recommender systems view the recommendation process as a static procedure disregarding the fact that users' preferences are changed over time. Reinforcement learning (RL) approaches are the most advanced and recent techniques used by researchers to handle challenges where the user's interest is captured by their most recent interactions with the system. However, most of the recent research on RL-based recommender systems focuses on simply the user's recent interactions to generate the recommendations without taking into account the context of the user in which these interactions occur. The context has a great impact on users' interests, behaviors, and ratings e.g., user mood, time, day type, companion, social circle, and location. In this paper, we propose a context-aware deep reinforcement learning-based recommender system focusing on context-specific state modeling methods. In this approach, states are designed based on the user's most recent context. In parallel, a list-wise version of the context-aware recommender agent is also proposed, in which a list of items is recommended to users at each step of interaction based on their context. The findings of the study indicate that modeling users' preferences in combination with contextual variables improves the performance of RL-based recommender systems. Furthermore, we evaluate the proposed method on context-based datasets in an offline environment. The performance in terms of evaluation measures optimally indicates the worth of the proposed method in comparison with existing studies. More precisely, the highest Presicion@5, MAP@10, and NDCG@10 of the context-aware recommender agent are 77%, 76%, and 74% respectively.

Keywords Recommender systems · Sequential decision making · Deep reinforcement learning · User's context

✉ Muazzam Maqsood
muazzam.maqsood@cuiatk.edu.pk

[1] Department of Computer Science, COMSATS University Islamabad, Attock Campus, Attock, Pakistan

[2] Computer Engineering Department, Sivas University of Science and Technology, Sivas, Turkey

1 Introduction

Information explosion goes much beyond gigabytes, exabytes, and petabytes to gigantic volumes in today's era of the digital world (Raza and Ding 2019). The evolution of Big Data is constantly shifting and serves as a primary motivating factor behind the trend of digitalization comprising Data Science, the Internet of Things, as well as Data mining (Oliverio 2018). The overall content and data are huge and cannot be managed by traditional database systems. Hence, we need a specialized tool to examine and extract useful patterns or information from them (Zikopoulos and Eaton 2011). On the one hand, increased data leads to greater convenience; but on the other hand, this poses the problem of retrieving useful information based on particular user interests and preferences (Vaidya and Khachane 2017; Sharma et al. 2021). In such challenges of Big Data, Recommender Systems (RS) are the systematic solutions for handling such extensive and complicated data as well as ranking the items depending on the user's preferences (Adomavicius and Tuzhilin 2005). The indications of these solutions are affirmed in several domains including e-commerce, e-learning, entertainment industry, travel, and tourism domains, as well as food and other items (Lu et al. 2015). Message filtering and information retrieval are the two early approaches used as a basis for the evolution of research in the domain of RS (Nichols et al. 1992). Following on, in the middle of the 1990s, the RS became an independent field as well as one of the hotspot topics among researchers that primarily depends upon rating behaviors (Adomavicius and Tuzhilin 2005) (Bi, et al. 2019; Frolov and Oseledets 2016; Kumar et al. 2022).

Popular traditional methods of recommender systems include collaborative filtering (Ghasemi and Momtazi 2021), content-based filtering (Pérez-Almaguer et al. 2021), hybrid methods(Dhruv et al. 2019), matrix factorization (D'Amico et al. 2022), and deep learning algorithms(Dong et al. 2022). These traditional recommenders model the user's preferences in terms of rating predictions. One of the major challenges in these methods is that they regard the recommendation as a static procedure ignoring the fact that users' interests are changed over time (Chen, et al. 2109; Liu, et al. 1810; Shani, et al. 2005). In a real scenario, at any timestamp T, a list of items is recommended to the user, and later on, the user provides feedback in terms of ratings or clicks. Depending upon this feedback, a new set of items is recommended to the user at the timestamp $T + 1$ that are most relevant to their current interest (Zhao et al. 2013). This will conclude that the essence of interactions of users with a recommender system is sequential (Zimdars et al. 1301), and the subject of generating optimal recommendations to a user involves a sequential decision problem as well as a prediction one (Shani, et al. 2005). Recently, RL methods have mostly been adopted methods in sequential-decision problems with a great pledge such as Gaming Industry (Lample and Chaplot 2017; Mnih, et al. 1312), ad bidding (Jin, et al. 2018; Wu, et al. 2018), and searching for optimal configuration of layers in neural networks (Jaques12, N.,, et al. 1611; Neary 2018). Currently, it has also been introduced in the field of recommender systems to design the intelligent recommender agent that adapts itself depending on the user's recent feedback (Huang et al. 2021). Researchers have employed different algorithms of reinforcement learning to carry out the recommendation challenge. Some of them include traditional deep-Q-networks and value-based methods in which the probability of taking each action is approximated followed by choosing the one having the highest Q-value. However, these methods are not suitable for an environment having millions of item spaces (Zheng, et al. 2018; Zhao, et al. 2018a). In addition, some studies have employed actor-critic-based mod-

els and in these, the actor inputs the states followed by states-specific scoring, and later on, this is fed into the critic model to approximate-Q-value (Liu et al. 2020; Zhao, et al. 2018b). Actor-critic models are more appropriate and have fewer temporal complications for problems with large action spaces (Zhao, et al. 2018b).

However, the existing deep-reinforcement learning-based recommender systems have one of the shortcomings is that the user's context is not considered while generating recommendations. Context is a broad phrase that includes location, surroundings, social networks, age, gender, history, emotion, time (morning, evening, midday), and other factors while rating items (Iqbal et al. 2019). To enhance user satisfaction and happiness, recommender systems should adhere to the user's context. This has led to the development of the CARS (Context-aware recommender systems) idea, a new kind of recommender system that aims to perform better (Adomavicius and Tuzhilin 2011). Generally, the recommendation techniques depend upon two dimensions $(users \times items)$ in which N is comprised of item space that is relevant to a particular user M (Adomavicius and Tuzhilin 2005). Users are indicated by $U = \{u_1, u_2 \ldots u_n\}$ in which n the total number of users interacting with the agent, as well as the set of items being recommended by agent are $I = \{i_1, i_2 \ldots i_m\}$ where m is the total number of items. Many, but certainly not all of the items have been rated by the users. Ratings are indicated by r_{iu}, wherein i and u is the collection of rated user-item pairings. However, in context-aware recommender systems, a new dimension of context has been added to generate the recommendations i.e. $(users \times items \times context)$. In this research, we have user $U = \{u_1, u_2 \ldots u_n\}$, items $I = \{i_1, i_2 \ldots i_m\}$ as well as the context of the user $C = \{c_1, c_2 \ldots c_t\}$ in which r_{iuc} shows the contextualized ratings. In the existing studies, a great deal of research is done on context-aware recommender systems using traditional algorithms, machine learning, and deep learning algorithms for rating predictions (Musto et al. 2021; Sattar and Bacciu 2022; Unger et al. 2020; Jeong and Kim 2021). Some specialized set of learning in recommender algorithms with context is also studied such as kernel mapping (Iqbal et al. 2019). Recently, graph-based learning algorithms have been developed as a specific collection of algorithms that incorporate the user's personality states and context (Musto et al. 2021; Sattar and Bacciu 2022). However, graph models also disregard the dynamic aspect of user's evolving interests with time.

From the above discussion, it is evident that in existing studies different context-aware recommender systems are proposed using different methods (Ghasemi and Momtazi 2021; Pérez-Almaguer et al. 2021; Dhruv et al. 2019; Iqbal et al. 2019; Musto et al. 2021; Sattar and Bacciu 2022, 2023). However, these methods do not have the capability to adapt to the evolving interests of users because these methods view recommendation as a static process. To address this, an adaptable method based on reinforcement learning is proposed and because of the dynamic nature of RL, the system can constantly learn and modify its recommendations according to feedback from users. Although other reinforcement learning methods are utilized to solve the recommendation problem (Zheng, et al. 2018; Zhao, et al. 2018a, 2021; Liu et al. 2020). However, the novel aspect of this study is that the user context is also taken into account when designing the recommendation algorithm utilizing reinforcement learning. A very generic flow of proposed contextualized interaction with RL-agent is depicted in Fig. 1. Moreover, in the proposed study, we have not modeled the context as a piece of embedded information into states rather we model the state based on the current context of the most recently clicked items by the user. More specifically, we model the states by designing the dynamic state's mechanism based on user context. In this work,

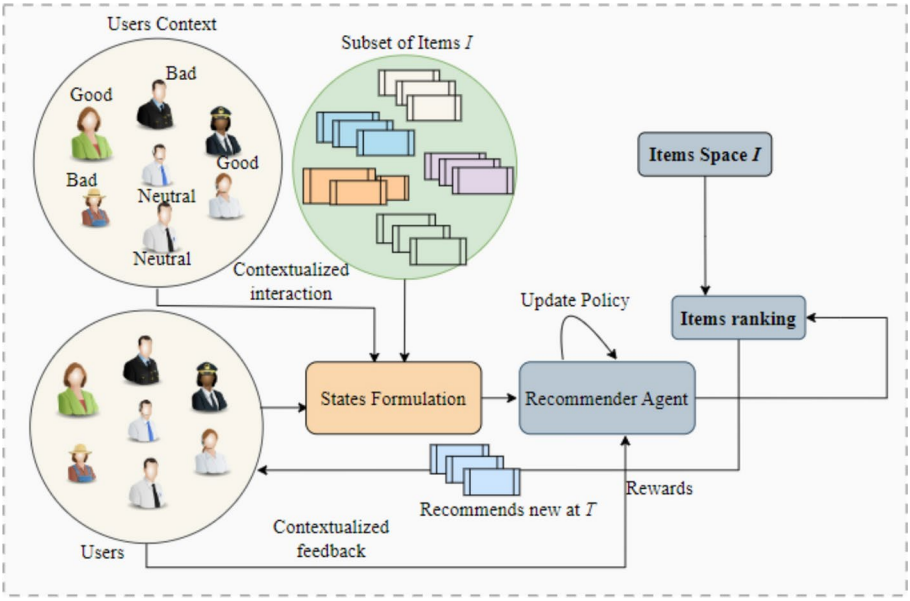


Fig. 1 Contextualized-User item interaction with the RL-agent

we propose two different strategies namely "Context-based zero weighting (*CsZW*)" and "Context-based Attention weighting (*CsAW*)". These weighting strategies assign weights to the subset of item space present in the user's most recently clicked items depending upon the user's context. Later, the weighted items are considered to be offered to the actor agent to decide which item to recommend in the next time step T. Following that, the critic models determine if the decision made by the actor while taking into account the users' context is either good for the current state or not. These modules capture users' most current as well as recent context interests. Furthermore, we designed context-aware list-wise recommender agents in which a list of things is recommended to the user depending on the previous list of items that the user likes in a particular situation. The proposed agent is evaluated on multiple reward functions, such as binary and 5-scale rewards, using different sampling methodologies of agent experience replays during training.

The following are the contributions of this study:

- A recommender agent based on an actor-critic algorithm of reinforcement learning is proposed to generate context-aware recommendations
- A unique approach of dynamic state modeling based on the user's most recent context is designed to capture users' interests and preferences in their specific context
- A list-wise context-aware agent is also proposed in which a context-specific list of items is recommended at each step of interaction

The rest of the paper is categorized into several sections: Section II describes the related work, section III explains the proposed framework, and Section IV presents the results followed by a conclusion.

2 Related work

The realm of recommender systems is relatively broad, and approaches to the recommender model fall into multiple categories. Those techniques include the conventional methods including collaborative filtering, content-based filtering, and hybrid models (Isinkaye et al. 2015). They are further classified as memory-based and model-based techniques. In model-based methods, machine learning and deep neural networks are also adopted. RL techniques are also utilized for recommendations. Some studies employ context, trust, personality attributes, and emotions as additional information to user-item interactions. (Iqbal et al. 2019; Rahim, et al. 2020; Dhelim 2022; Yousefian Jazi et al. 2021a). From the perspective of this work, we have divided the approaches into existing context-aware recommenders and RL-based recommenders.

The literature contains recommender agents that use classic approaches to exploit various sorts of contextual factors such as emotions, mood, and so on. More explicitly, these recommendation systems are either built using the traditional or deep learning approaches. For instance, Cui et al. (2018) exploited the context features by suggesting the two-level SVD technique. Their proposed model is also combined with post-context filtering to improve the recommendation quality. Furthermore, Iqbal et al. (2019) proposed kernel mapping-based context-aware recommender systems in which they studied different contextual variables such as user mood, social circle, time, etc. during the interaction with the system. Jeong and Kim (2021) proposed a deep-learning approach to context-aware recommendations. They employ the features of users, items, as well as context as input to the deep learning model based on a neural network and autoencoders to forecast the interests of the users. Sattar and Bacciu (2023) proposed a very effective method namely graph neural networks for context-aware recommendation in which a user-item opinion graph is designed with edges denoting the context of the user. Casillo et al. (2022) also proposed a context-aware recommender system by introducing the notion of embedded context. With the matrix factorization method, they have added the third dimension i.e. context in the matrix, in addition to ratings and users and attained good results. Taneja and Arora (2019) work on the prioritization of different contextual factors because integration of all context factors will lead to computational complexity and ultimately influence the recommendation results. To address this, they have designed the neural network in order to figure out the relevant contextual attributes. However, there is a need to design an improved message-parsing strategy in case users are categorized into multiple subgraphs. Li et al. (2018) proposed a multi-dimensional context-aware recommendation method depending upon the enhanced random forest technique. This improved model computes the weights of the 3-D user-item-context model and provides recommendations to users with the same context using items that have high prediction ratings. Unger and Tuzhilin (2020) extend the technique of collaborative filtering by employing structured as well as unstructured latent contextual information. Their finding indicates that context-aware recommendations are improved if hierarchical latent contextual representations are employed. Yousefian Jazi et al. (2021b) also proposed an emotion-aware music recommender, in which the user's keystrokes as well as patterns of mouse clicks are taken into account. They have employed the EMA-EMA method that is reliant on collaborative filtering. In their technique, users' similarity is computed depending on how they interact with mouse and keyboards. Another interesting recommender system is proposed in the work of Rostami et al. (2022) in which a hybrid model of graph clustering and deep learning

is designed for time-aware food recommendation. In terms of precision, recall, and NDCG, their proposed method shows state-of-the-art performance over existing methods. In addition to contextual factors, Guo et al. (2019) proposed a collaborative filtering approach in which they combined contextual factors of emotion with trust data. Objective trust among users is determined by the resemblance of opinions involving similarities between ratings and preferences. Zheng (2022a) carried out an empirical study in which context-aware collaborative filtering methods are contrasted on different datasets. They have highlighted that context-aware datasets are usually sparse since the user does provide a rating in every context. Exploiting the contextual variables with traditional and deep learning methods shows good performance in comparison with recommendations without context, but these methods do not perfectly capture the user's dynamic and evolving interests in a different context. However, these systems have a significant limitation in that they view recommendation as a static process and are unable to adapt to users' constantly evolving preferences. Similarly, in Zheng et al. (2018) only item context is exploited using RL. They did not consider the user context while recommending news. Therefore, it is necessary to investigate their influence using a sequential decision-making algorithm like RL, however, there is not any research that involves user context e.g. user mood, and emotions, while making recommendations. In the proposed study, we contribute to the literature by combining the user's context with the reinforcement learning approach, which is more flexible to users' changing interests and contextual factors.

Reinforcement learning algorithms provide a revolutionary shift to existing methodologies of recommender systems. In some recent studies, several RL-based agents are designed to improve the recommendation performance. They are further subdivided into value-based models, such as deep-Q-Networks, and policy-based models. For instance, Zheng et al. (2018) suggest a DRL-based recommender agent for making news recommendations. The proposed reinforcement learning model acquires the user's interest as feedback. This feedback is regarded as a reward function, and it is either clicks or rating information. Similarly, Hu et al. (2017) designed the recommender agent using DRL for the music domain. In their proposed method, incentives are computed by collecting data from users who click, listen, or download the music. Following on, Lei and Li (2019) design the user-specific states by utilizing the matrix factorization technique. The underlying DRL framework in their model is based on deep-Q-learning and experimentation is performed with two different protocols i.e., ignoring the missing ratings and considering the missing ratings as negative feedback. Furthermore, Zhao et al. (2021) designed the deep-Q-network-based agent for the task of online advertising. Their suggested model determines the three different tasks simultaneously, such as whether the advertisement should be interpolated into the recommendation list or not. Another study by Zhao et al. (2018a) in which a novel Deep-Q-Network model is proposed also referred to as DEERs. In this model, two different states are designed to balance the positive and negative feedback for an agent. One state contains the items that users have clicked, while the other contains the items that users have skipped. The balanced feedback mechanism helps in improving the performance of the recommender agent. All of these studies employ the deep-Q-learning framework of reinforcement for designing the recommender agent. However, the deep-Q-Networks takes states as input and computes the Q-value for every state-action pair which will lead to the temporal complexity of the algorithm. Another set of studies is also suggested by various researchers in which recommender agents are designed using the actor-critic model. For instance, Zhao et al. (2018b) proposed

the page-wise recommender system. In their proposed algorithm, the agent recommends the page of items and receives simulated feedback. Similarly, Liu et al. (2020) proposed the state-representation modules to capture the user's preferences more accurately with an agent based on an actor-critic framework. Although these approaches provide good results, they have limitations in that they do not account for the context in which the user interacts with the system, such as user emotions.

3 Methodology

A detailed step-by-step explanation of the proposed work is presented in this section. The theoretical framework is added first, and then an explanation of the architecture and working of the proposed recommender agent is presented. The mathematical notations are provided in Table 1.

In this work, the recommendation problem can be represented as a Markov Decision Process (MDP), in which the recommender (i.e., agent) interacts with users (i.e., considered as environment) to progressively recommend a set of items according to the context of the user. An actor-critic agent can be used to implement this concept of context-aware Markov Decision Process (MDP). Mathematically, this can be formulated as follows:

$$A \leftarrow E : (U \times I \times C) \in U_i \therefore Envionment \tag{1}$$

where
$I_i = \{i_1, i_2, i_3 \ldots i_N\}\, I_i \subset I$ and $\therefore A \epsilon ActorCritic$ (2).

In the above Eq. (1–2), the agent based on the actor-critic framework perceives the environment in which different users $U = \{u_1, u_2 \ldots u_n\}$ interacts with items $I = \{i_1, i_2 \ldots i_m\}$ in a specific context $C = \{c_1, c_2 \ldots c_t\}$ and it recommends another subset of items I to the users U. A more detailed formulation is given in the next Sect. 3.1.

Table 1 List Of Notations

Notations	Meaning
$U = \{u_1, u_2 \ldots u_n\}$	Set of users
$I = \{i_1, i_2 \ldots i_m\}$	Set of items
$C = \{c_1, c_2 \ldots c_t\}$	Set of contextual variables
r_{iuc}	Contextualized ratings
s_t	State of user at time t
$(S, A, \mathcal{P}, \mathcal{R}, \gamma)$	MDP model
E_{NCF}	Explicit neural collaborative filtering
$\{i_{e1}, i_{e2}, i_{e3} \ldots i_{eN}\}$	Item embedding's
C_{st}	Context specific state
C_{a_t}	Context specific action
a_t	Action at time step t
p_{θ^π}	Actor network
$u_{im=}\{g_1, g_2, g_3, \ldots . g_m\}$	User embedding's

3.1 Problem formulation with Markov decision process (MDP)

In the first stage, the formulation of context-aware recommendation is conceptualized. For this, consider a set of users U and an item space$I$, where R is a collection of ratings, i.e. User u_i provides rating r_{ui} to the ith item. The interaction of the recommender agent with the environment E (or users) using reinforcement learning is depicted in Fig. 1. The recommender agent sequentially performs a ranking of the items from item space I over a series of time steps T with the objective of increasing the cumulative reward in the form of feedback from users. More precisely, at each time step T, the recommender agent perceives the state $s_t \in S$ of user $u_i \in U$ followed by taking action $a_t \in A$ (item) from the whole item space I and receiving a reward R in form of clicks or ratings. Later on, the state of the user u_i is changed to $s_{t'}$ with a transition $P(s_{t'}|s_t, a_t)$ and a goal to determine the optimal policy $(\pi_\theta : S \times A \rightarrow [0,1]$. The constituents of the MDP i.e., $(S, A, \mathcal{P}, \mathcal{R}, \gamma)$ for the proposed model with contextual-variables is given below:

States*S*: A state is defined as the user's history of recently clicked items i.e.,$s_t = \{i_1, i_2, i_3 \ldots i_N\}$ as well as the context i.e., $c_t = \{c_{t_1}, c_{t_2}, c_{t_3} \ldots c_{t_n}\}$ in which these items are clicked. The context $c_t \in C$ where C is the set of contextual variables. The items $\{i_1, i_2, i_3 \ldots i_N\}$ in s_t is the learned embedding from the explicit neural collaborative filtering algorithm E_{NCF}. Mathematically, the state sampling is given in Eqs. (3-5):

$$s_t = \{i_1, i_2, i_3 \ldots i_N\} \rightarrow E_{NCF} \rightarrow \{i_{e1}, i_{e2}, i_{e3} \ldots i_{eN}\} \tag{3}$$

$$c_t = \{c_{t_1}, c_{t_2}, c_{t_3} \ldots c_{t_n}\} \rightarrow f(n^{th}C_t) \rightarrow \{c_{tm_n}\} \tag{4}$$

$$C_sZW|C_sAW\left(\{i_{e1}, i_{e2}, i_{e3} \ldots i_{eN}\}, \{c_{m_1}, c_{m_2}, c_{m_3} \ldots c_{tm_n}\}\right) \rightarrow C_{st} = \{C_{i_1}, C_{i_2}, C_{i_3}, \ldots . C_{i_n}\} \tag{5}$$

More precisely, in Eqs. (3–5), state s_t is sampled from the users u_i recent positive interaction history followed by applying the E_{NCF} to get embedding items i.e. $\{i_{e1}, i_{e2}, i_{e3} \ldots i_{eN}\}$. Subsequently, the $f(n^{th}C_t)$ function, will return the context-specific element $\{c_{tm_n}\}$ based on which $C_sZW|C_sAW$ module will compute the context-specific state $C_{st} = \{C_{i_1}, C_{i_2}, C_{i_3}, \ldots . C_{i_n}\}$.

Actions*A*: An action a denotes the item from item space I represented as a continuous parameter vector i.e.,$a_t \in \mathbb{R}^{1 \times P}$. The inner products of the action a_t from the actor, models are performed with item embedding[1] of items in I that is used to compute the ranking scores of all items. When an action is done, the item with the highest ranking is recommended to a user u_i. Mathematically, the action A is formulated in Eq. (6–8):

$$A :\rightarrow C_{st} = \{C_{i_1}, C_{i_2}, C_{i_3}, \ldots . C_{i_n}\} \tag{6}$$

$$\forall(e_i \in I) \rightarrow (e_i).(_{a_t})^T \rightarrow f_s \therefore scores \tag{7}$$

$$f_s \rightarrow Ranking \rightarrow R_t \rightarrow \max(e_i \in I) \tag{8}$$

In the above Eq. (6–8), C_{st} is the input of the actor-model which will result in state-specific a_t. The dot product is taken among the transpose of $a_t \rightarrow {a_t}^T$, and each embedding of items

[1] Item embedding of all items in item space I is learned by Explicit Neural Collaborative filtering (NCF).

presents in I. The item with the highest-ranking score $Ranking \rightarrow R_t$ is then selected as an item to be recommended to the user.

State Transitionsρ: When the feedback of the users in the form of a rating with explicit context $c_t \in C$ is provided, the state transition is performed$(s_{t'}|s_t, a_t)$. This transition is only conceivable when the feedback is positive since the s_t should reflect the user's positive interaction history with context. Mathematically, the state transition is given in Eq. (9):

$$C_{st} = \begin{cases} C_{st} = \{C_{i_1}, C_{i_2}, C_{i_3}, \ldots . C_{i_n}\} \rightarrow C_{st} = \{C_{i_1}, C_{i_2}, C_{i_3}, \ldots . C_{i_n}, C_{i_p}\} \, if R\left(C_{st}, C_{at}\right) > 0 \\ C_{st} = \{C_{i_1}, C_{i_2}, C_{i_3}, \ldots . C_{i_n}\} \rightarrow C_{st} = \{C_{i_1}, C_{i_2}, C_{i_3}, \ldots . C_{i_n}\} \, otherwise \end{cases} \tag{9}$$

In the above Eq. (9), it is illustrated that the context-aware user state C_{st} is only updated if the feedback $R\left(C_{st}, C_{at}\right) > 0$ is positive and a new item is added at the end i.e. C_{i_p}.

Rewards R: Given the user state s_t and the action a_t, recommendations are provided by the agent. Later on, the feedback in the form of clicks, ratings, with context is provided to the agent. Through this feedback, the agent obtains an instant reward $R(s_t, a_t)$. Mathematically, the reward formulation is given in Eq. (10–11):

$$A :\rightarrow C_{st} = \{C_{i_1}, C_{i_2}, C_{i_3}, \ldots . C_{i_n}\} \rightarrow a_t \tag{10}$$

$$a_t \rightarrow U_i \rightarrow feedback(R\left(C_{st}, C_{at}\right), \{c_{tm_n}\}) \tag{11}$$

In the above Eq. (11), when the agent takes an action a_t on $C_{st} = \{C_{i_1}, C_{i_2}, C_{i_3}, \ldots . C_{i_n}\}$ it receives feedback in the form of a rating according to a scale of 1–5.

Discount Factor γ: The value of the discount factor lies in between $\gamma \in [0,1]$ through which the current value of long-term rewards is computed.

With the above-listed problem formulation, the task of context-aware recommendation is typically conceptualized as follows: Given the context-aware historical MDP, i.e.$(S, A, \mathcal{P}, \mathcal{R}, \gamma)$, the objective is to approximate the policy of recommendation $\pi_\theta : S \rightarrow A$ that will increase the cumulative reward with specific context while making recommendations.

3.2 Deep reinforcement learning (DRL) framework

In the proposed model, the actor-critic-based DRL framework is employed to achieve the context-aware recommendation. The strategies are proposed for context-specific state modeling in which the policy of recommendation is learned via an actor model while the critic is based on value function to learn the optimal Q-values. The proposed is divided into different parts i.e., we built the actor-critic model first, and then we proposed the methods for modeling context-specific states known as "Context-specific Zero weighting" C_sZW and "Context-specific Attention weighting" C_sAW.

3.2.1 Actor critic model

The actor-critic model is a reinforcement learning model in which the actor is a policy network while the critic learns the value-based function. It is one of the temporal difference variants of policy gradient. More precisely, the actor model decides the action that should

be taken while the critic evaluates the quality of the action taken as well as how it should be adjusted. Actor-critic models are trained in a game-like session, just like Generative Adversarial networks in which the generator model generates data to fool the discriminator while the discriminator improves its capability to correctly identify the generated and real data. It efficiently tackles the challenges of high variance which often happen in the back-propagation of policy-based methods. Combining actor-critic techniques with generalized advantage estimation significantly reduces the variability of gradient changes.

Figure 2 shows the working of the actor-critic model. The input of the actor model is the user's recently clicked items as $s_t = \{i_1, i_2, i_3 \ldots i_N\}$ where this state s_t is modeled using "Context-specific Zero weighting" $C_{stp}(w_0)$ or "Context-specific Attention weighting" $C_{stp}(w_{atten})$. In addition, this state captures the user's current preferences based on the contextual variables $c_t \in C$ at time step T_n where n is the user's last or n^{th} interaction with the system. At a particular time step T, the state of the user $s_t = \{i_1, i_2, i_3 \ldots i_N\}$ is passed through $C_{s_t} = C_{stp}(w_0)$ or $C_{s_t} = C_{stp}(w_{atten})$ where C_{s_t} is the context-specific state comprising of item-embedding i.e., $C_{s_t} = \{C_{i_1}, C_{i_2}, C_{i_3}, \ldots . C_{i_n}\}$. The dimensions of the C_{i_j} is k-dimensional vector of the form $C_{i_j} \in \mathbb{R}^{1\times k}(i_j \in 1 \ldots . . n)$.

$$p_{\theta^\pi} : C_{s_t} \rightarrow C_{a_t}$$

Following on, the resulting C_{s_t} is passed through two fully connected layers in the actor model having activations ReLu. Subsequently, the last layer having a dimension equal to the state size is added to generate the action a_t. This step maps the context-specific state C_{s_t} to a context-specific action C_{a_t} by an actor model denoted as p_{θ^π} as shown in Eq. (12):

$$p_{\theta^\pi} : C_{s_t} \rightarrow C_{a_t} \tag{12}$$

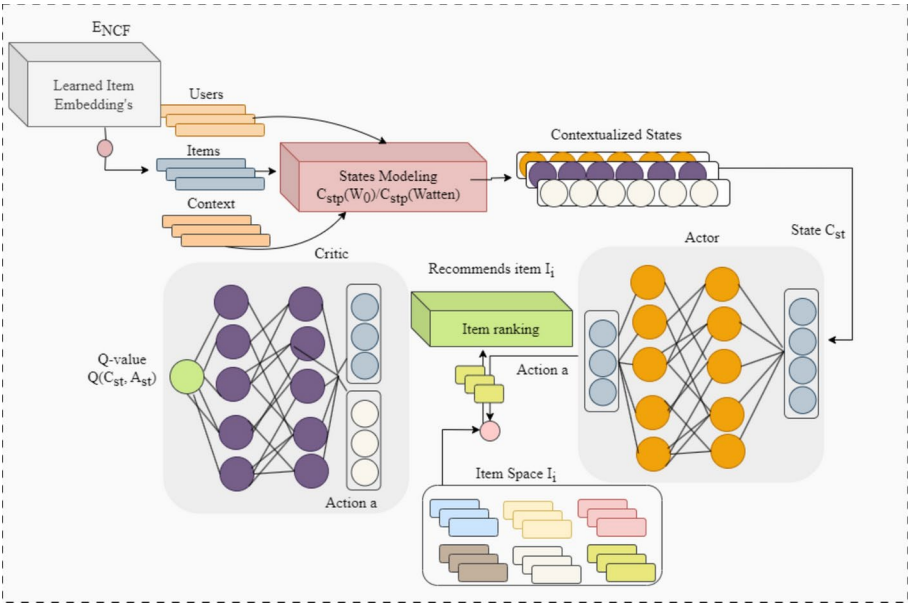


Fig. 2 A pictorial overview of the Actor-Critic model

In the above equation, where p_{θ^π} is the policy network parameterized by θ^π to transform the state vectors into action vectors. In the following step, the ranking scores of all items present in the item space I are computed based on the context-specific action C_{a_t}. The embedding of the items in I is linearly correlated with $C_{a_t} = p_{\theta^\pi}(C_{s_t})$ to generate the item for recommendation as $a_t \in \mathbb{R}^{1\times P}$. The ranking scores of all items $i \in I$ are computed by Eq. (13):

$$R_{scores} = dot\left\{(e_i \in I).(C_{a_t})^T\right\} \tag{13}$$

The set of ranking scores denoted as R_{scores} is used to choose the item for recommendation having the highest score. In addition, the agent is not allowed to choose the same item in one single round of interactions i.e., at each time step the previously recommended item is discarded from the item space *I*.

Following on, we have designed a critic-model G that is simply a deep-Q-Network that performs the approximation of state-action value pairs $Q(C_{s_t}, C_{a_t})$ to determine whether or if the action chosen by the actor-model is relevant to the user's present state in a given context. More precisely, the input of the critic-model is a user context-specific state modeled from either $C_{stp}(w_0)$ or $C_{stp}(w_{atten})$ as well as the C_{a_t} generated by $p_{\theta^\pi}(C_{s_t})$. In addition, the output of the critic-model is a learned Q-value regarding different items. Then based on this value, the parameters of the actor model p_{θ^π} are updated with the aim of better context-aware recommendations in a future round of interactions. The update in the actor-framework is accomplished through context-specific sampled policy gradient given in Eq. (14):

$$\begin{aligned}\nabla_\theta G(p_{\theta^\pi}) \cong \frac{1}{M}\sum_t \nabla_a Q(C_{s_t}, C_{a_t}) \\ |_{\{C_{s_t}=C_{stp}(w_0) or C_{stp}(w_{atten})\},\{a=C_{a_t}=p_{\theta^\pi}(C_{s_t})\}} \\ \nabla_\theta \pi^\theta(C_{s_t})\,|_{C_{s_t}=C_{stp}(w_0) or C_{stp}(w_{atten})}\end{aligned} \tag{14}$$

In the above equation, $G(p_{\theta^\pi})$ is the estimate of all feasible Q-values resulting from the policy θ^π while the M represents the size of the batch sampled from replay memory. The critic-model is updated by reducing the errors as a loss function $E(\theta^c)$ given below in Eq. (15):

$$E(\theta^c) = \mathbb{E}_{C_{s_t}, C_{a_t}, r_t, C_{s_{t+1}}}\{(y_t - Q(C_{s_t}, C_{a_t}; \theta^c))\}^2 \tag{15}$$

In the above Eq. (4), $y_t = \mathbb{E}_{C_{s_{t+1}}}[r_t + \gamma Q^*(C_{s_t+1}, C_{a_t+1}; \theta^{c^*})|_{C_{s_{t+1}}, C_{a_{t+1}}}$ is the label value for the current interaction time step T in which the r_t is the contextual ratings γ denotes the discount factor. In this study, the approach of the target network as in Liu et al. (2020) in which weights of the target actor-critic model denoted as w^* and θ^*. The working of actor-critic in pictorial form is given in Fig. 3.

3.2.2 Context-specific modeling of states

It has been observed in the literature section that context has a great impact on users' rating behaviors and also it has been exploited in many methodologies. It helps in improving the

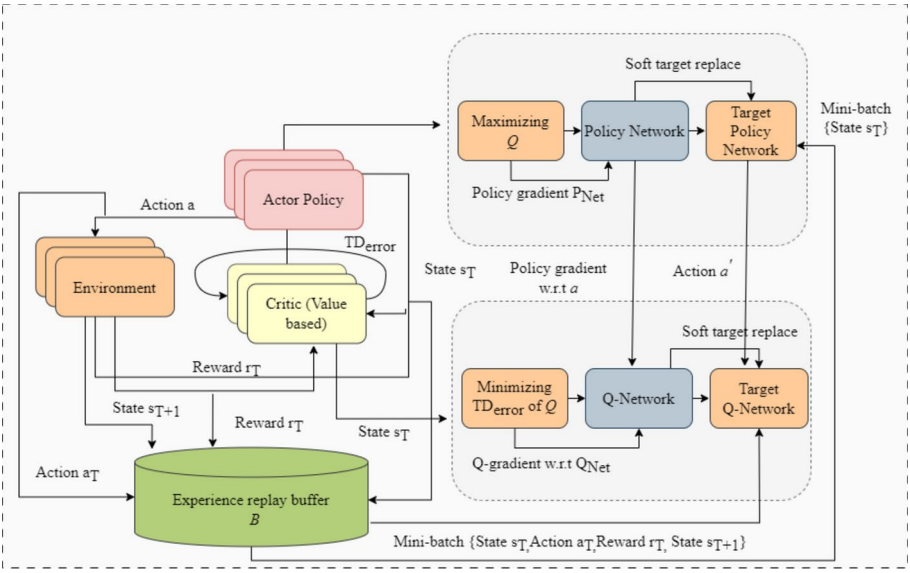


Fig. 3 An internal working of actor-critic architecture

performance of recommender systems by making recommendations depending upon the context of the users such as user mood and emotions, etc. However, it is necessary to design a suitable mechanism to capture the states of the users relative to contextual variables. To the best of our knowledge, no similar works exist in the associated RL-based recommendation in which states are modeled through contextual variables, and learning is made through the policy-value method. However, when it is solved via actor-critic then during item ranking, this produces noise since all item space embedding lacks context and is entirely dependent on the user's context when consuming an item. To handle such limitations and improve the performance of the recommender system with contextual variables with reduced temporal complexity, we propose two context-specific strategies for context-specific modeling of states. The first one is the zero-weighting strategy (C_SZW) and the second one is the attention weighting strategy (C_SAW). The generic formulation of context includes 3-dimensional data from the environment E including the embedding of user U_i i.e., $u_{im=}\{g_1, g_2, g_3, \ldots .g_m\}$, embedding's of the items I i.e., $\{i_1, i_2, i_3 \ldots i_N\}$ and the corresponding contextual variables i.e., $C_T = \{C_1, C_2, C_3, \ldots .C_n\}$. All of these become the input of the C_SZW and C_SAW to generate a contextualized state $C_{s_t} = \{C_{i_1}, C_{i_2}, C_{i_3}, \ldots .C_{i_n}\}$. Mathematically, Eq. (16) shows the formulation:

$$\begin{aligned} &C_sZW|C_sAW \leftarrow input(u_{im=}\{g_1, g_2, g_3, \ldots .g_m\}, \\ &I = \{i_1, i_2, i_3 \ldots i_N\}, C_T = \{C_1, C_2, C_3, \ldots .C_n\} \end{aligned} \tag{16}$$

$$C_sZW|C_sAW \rightarrow C_{s_t} = \{C_{i_1}, C_{i_2}, C_{i_3}, \ldots .C_{i_n}\} \tag{17}$$

In the following sections, a detailed discussion is added for both of the context-specific modeling of the states.

3.2.2.1 Context-specific zero weighting with dynamic states (CSZW) In the proposed context-specific zero-weighting $C_{s_t} = C_{stp}(w_0)$, the context is modeled from a state s_t by using the context of the user at T_n time step i.e., the context of the interaction at the nth item of the state s_t.

More precisely, the state $s_t = \{i_1, i_2, i_3 \ldots i_N\}$ of user u_i is sampled from the user's historical interactions. This state captures the user's recent interest indicated by the user's recently clicked items. Similarly, the context vector $C_T = \{C_1, C_2, C_3, \ldots . C_n\}$ is also sampled indicating the interactions of the given s_t in particular context $c \in C$. Following, the context at n^{th} time is acquired and the basis on that weights are assigned to the item in the user's recent state i.e., $s_t = \{i_1, i_2, i_3 \ldots i_N\}$. The items in state s_t that are not rated or liked in context at n^{th} time interaction is assigned with weights 0 since they are not related to the user's interest in their most recent context. Following on, the remaining items in s_t that are liked by the user at their context of nth time interaction are selected i.e.,$s^*{}_t$ and the embedding's $f = \{f_1, f_2, f_3, \ldots . f_n\}$ is transformed into the weight vectors by passing them into the 1D convolution layer resulting in another vector $(Q = Conv(w_i f)|_{1,2\ldots\ldots n})$. This new vector is the context-specific weighting of the items in the state s_t which is then fused with user embedding $u_{im=}\{g_1, g_2, g_3, \ldots . g_n\}$ whose context is computed to a vector depicting the context-specific user-item interactions. In the last, the context-specific vector Q, user embedding's u_{im}, and the fusion of both as $\mathcal{F}(Q, u_{im,}, Conv(w_i f)|_{1,2\ldots\ldots n})$ is combined to generate the $C_{s_t} = \{C_{i_1}, C_{i_2}, C_{i_3}, \ldots . C_{i_n}\}$. Mathematically, the context-specific zero weighting strategy is described in Eq. (18–20):

$$s^*{}_t = \{i_1, i_2, i_3 \ldots i_N\} \rightarrow \{w_1, w_2, w_3 \ldots w_n\} \therefore n^{th}(c) \tag{18}$$

$$Q = s^*{}_t = s^i{}_{t_j} \rightarrow \left[b_j{}^i + \sum_{k=1}^{N_{i-1}} Conv(w^{l-1}{}_{kj}, d^{i-1}{}_k) \right] \tag{19}$$

$$C_{s_t} = \{C_{i_1}, C_{i_2}, C_{i_3}, \ldots . C_{i_n}\} \rightarrow \mathcal{F}\left[(u_{im=}\{g_1, g_2, g_3, \ldots . g_n\} \times Q)\{u_{im}\}\{Q\}\right] \tag{20}$$

In the above Eq. (20), the $\{C_{i_1}, C_{i_2}, C_{i_3}, \ldots . C_{i_n}\}$ is the context-specific states of the users based on their context at the nth time interaction with the recommender agent. The pictorial representation of the context-specific zero-weighting strategy is depicted in Fig. 4. However, using the above formulation, the state's size does not need to be set but rather may be variable, introducing the dynamic state sizes mechanism in the recommender model.

3.2.2.2 Context-specific attention weighting (CSAW) In the above context-specific zero-weighting strategy, the items in the states s_t that are not related to the user's recent context are assigned zero weights which will diminish their role in generating action C_{a_t} by the parameterized function of the actor $p_{\theta^\pi}(C_{s_t})$. This introduces the issue of users favoring items that are very specific to their current context, reducing the diversity of the recommendation. To cope with such issues, we introduce the attention-based weighting strategy in which weights are assigned to all state items based on the subset of state items related to the user's interests and preferences with context at n^{th} time interaction with the agent. For this, consider the user's state $s_t = \{i_1, i_2, i_3 \ldots i_N\}$ and associated interactions with

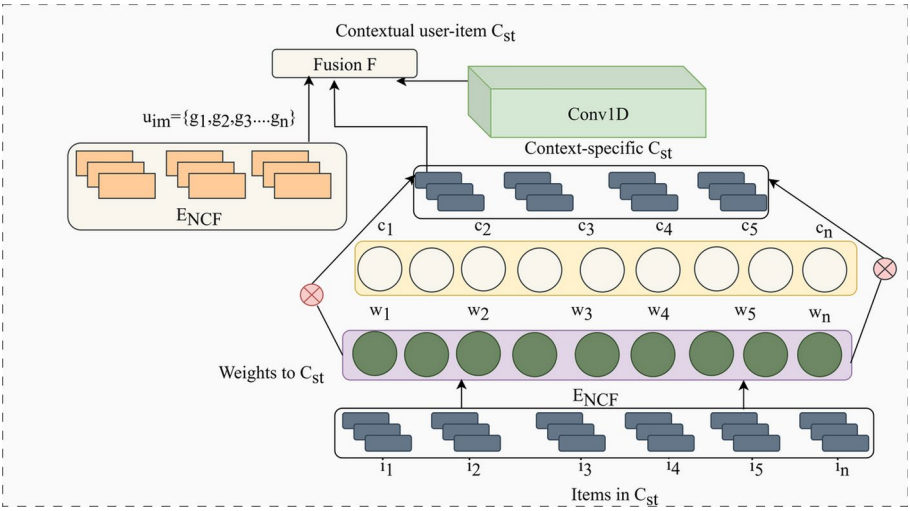


Fig. 4 A pictorial representation of C_SZW-based state-specific modeling

context vector $C_T = \{C_1, C_2, C_3, \ldots .C_n\}$. Following on, the context of the interaction at the nth item of the state s_t is chosen to generate the subset of the state s_t denoted as $(s^*{}_t = \{i_1, i_2, i_3 \ldots i_N\} \in c_{nth})$. The subset size of the state $s^*{}_t$ does not have to be fixed and can vary. The embedding of the state $(s_t = \{f_1, f_2, f_3, \ldots . f_n\})$ is transformed to weights vectors by employing the convolution layer resulting in a $(P = Conv\,(w_i f)\,|_{1,2\ldots\ldots n})$. Similarly, the subset state has item embeddings $s^*{}_t = \{S_1, S_2, S_3, \ldots . S_n\}$ is also transformed into weight vectors through a 1D convolution layer resulting in a vector $Q = Conv\,(w_i S)\,|_{1,2\ldots\ldots n}$. This step will introduce the two vectors where the first vector captures the current interests of the users in different contexts whereas the second vector shows the users' current interest in the most recent context i.e. context at n^{th} time interaction. Later on, the attention scores are computed by employing the attention layer i.e., Luong-style attention (Luong et al. 1508). For this, $P = Conv\,(w_i f)\,|_{1,2\ldots\ldots n}$ is treated as key vectors while $Q = Conv\,(w_i f)\,|_{1,2\ldots\ldots n}$ is treated as query vector. The dimension of both vectors is represented as d_p and d_q. The dot product of both vectors followed by applying the softmax is then used to determine the weights of all items in the state of users' current interest in a certain context. These resulting weights vector $Atten_{scores} = \{w_1, w_2, w_3, \ldots . w_n\}$ will provide the context-specific importance to all items in the user's current state s_t . The resulting vectors are then fused with users' embedding's $u_{im=}\{g_1, g_2, g_3, \ldots . g_n\}$ to generate the context-specific interactions of a particular user in the form of vector Q . In the last, the context-specific vector Q, user embedding's u_{im}, and the fusion of both as $\mathcal{F}\,(Q, u_{im,}, Atten_{scores})$ is integrated to generate the $C_{s_t} = \{C_{i_1}, C_{i_2}, C_{i_3}, \ldots . C_{i_n}\}$. Mathematically, the context-specific attention weighting is described in Eqs. (21–25) and pictorially given in Fig. 5:

$$s_t = \{f_1, f_2, f_3, \ldots . f_n\} \text{ and} s^*{}_t = \{S_1, S_2, S_3, \ldots . S_n\} \therefore n^{th}(c) \tag{21}$$

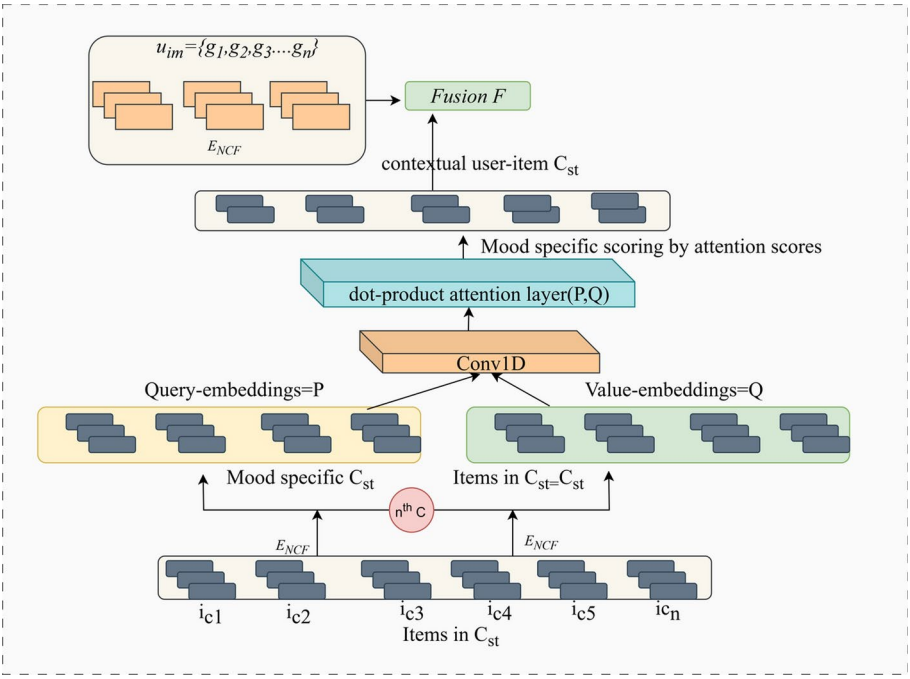


Fig. 5 A pictorial representation of C_SAW-based state-specific modeling

$$P = s_t = s^i{}_{t_j} \rightarrow \left[b_j{}^i + \sum_{k=1}^{N_{i-1}} Conv(w^{l-1}{}_{kj}, d^{i-1}{}_k) \right] \tag{22}$$

$$Q = s^*{}_t = s^i{}_{t_j} \rightarrow \left[b_j{}^i + \sum_{k=1}^{N_{i-1}} Conv(w^{l-1}{}_{kj}, d^{i-1}{}_k) \right] \tag{23}$$

$$Atten_{scores} = \{w_1, w_2, w_3, \ldots . w_n\} \leftarrow dotproducttattentionLayer(P, Q) \tag{24}$$

$$C_{s_t} = \{C_{i_1}, C_{i_2}, C_{i_3}, \ldots . C_{i_n}\} \rightarrow \mathcal{F}\left[(u_{im=}\{g_1, g_2, g_3, \ldots . g_n\} \times Atten_{scores})\{u_{im}\}\{Atten_{scores}\}\right] \tag{25}$$

In the above Eq. (25), the $C_{s_t} = \{C_{i_1}, C_{i_2}, C_{i_3}, \ldots . C_{i_n}\}$ represents the users' context-specific states during the nth encounter with the recommender agent.

3.3 Training of context-aware RL agent

In this study, the proposed recommender agent is trained using the deep deterministic policy gradient (DDPG). The training of the algorithm starts by randomly sampling the users from the training set. Following on, the state of that particular user u_i is sampled from its historical interactions and passed from context-specific modeling to generateC_{s_t}. This state C_{s_t} is given as an input to the actor model to generate an action $C_{a_t} = p_{\theta^\pi}(C_{s_t})$ with ε-greedy exploration followed by making a recommendation by prioritizing the item

space I depending upon the ranking scores. Following that, by using the offline logs the reward r_t of the recommended item is computed in the form of either rating or clicks. If the reward of the recommended item is positive, then state transition is performed by adding the recommended item into the user stateC_{s_t}. In the last, the replay memory M is updated to include$< C_{s_t}, C_{a_t}, r_t, C_{s_{t+1}}) >$. Subsequently, the recommender agent selects the N samples from the replay memory to modify the weights of both actor and critic models by utilizing the Eq. (14). The whole algorithm runs for a fixed number of episodes where each episode lasts for T time steps. Furthermore, the actor model consists of an input layer of the same size as the embedding dimensions, followed by two hidden layers with the activation function ReLu and hidden units of 128. In the last fully connected layer, hidden units equal to the embedding dimension of states with an activation function "tanh" are added. The model contains a total of 67,940 trainable parameters and a size of "265.39 KB". Similarly, the critic model has an input layer and two hidden layers with 128 hidden units. Finally, a fully connected layer with a hidden unit of 1 and "linear" activation is added. The critic model has 72,469 trainable parameters and is "283.08 KB" in size.

4 Experiments

We have discussed the findings of the suggested context-aware recommender agent. In addition, we have provided information on the datasets used in this work.

4.1 Evaluation criteria

After training, the recommender agent is put into the test mode to evaluate its performance. We have followed the offline evaluation as a primary evaluation criterion that is suggested in Liu et al. (2020). In this kind of evaluation procedure, the model parameters are not updated during an interactive session with the users. More precisely, in an offline evaluation, the item space I available for the agent to recommend items is limited to the items present in the current session. The logic behind this technique is that data regarding ratings or click feedback is only available for items in this session. This feedback is retrieved through offline logs of the user's previous interactions with the system. As suggested in Liu et al. (2020), the offline evaluation is the ranking of the items available in the current session in a repeated manner in relation to the action provided by the Actor-network. Furthermore, in this study, the recommender agent is allowed to recommend only one item in one time step T_i, therefore precision, MAP, and other scores are computed at the end of the session. In addition, we have also performed the list-wise testing of suggested context-aware recommender agent with different replays and reward criteria. Moreover, we have considered the precision, recall, MAP, and NDCG (Normalized Discounted Cumulative Gain) as evaluation metrics to indicate the performance of the proposed method. The mathematical formulae used to compute them are as follows:

4.1.1 Precision:

Precision is defined as the likelihood of identifying relevant items from the available items recommended to the user by the recommendation system (Iqbal et al. 2019).

The precision@k, on the other hand, is the ratio of the relevant items in the topK items of recommendations lists. Items are relevant if their rating is more than 3; otherwise, they are irrelevant. To categorize the items as relevant or irrelevant, we utilized a threshold of 3 on a 1–5 rating scale. Mathematically, precision is computed by Eq. (26):

$$Precision = \frac{Items_{relevant}}{Items_{total_selected}} \tag{26}$$

4.1.2 Recall

The likelihood of identifying relevant items from a set of all relevant items is referred to as recall (Iqbal et al. 2019). The Recall@k, on the other hand, is the ratio of the relevant items from a set of all topK relevant items. Items are considered relevant if they have a rating greater than 3; or else they are irrelevant. We used a threshold of 3 on a 1–5 rating scale to determine whether an item is relevant or irrelevant. It is expressed mathematically as follows in Eq. (27):

$$Recall = \frac{Items_{relevant}}{Items_{total_relevant}} \tag{27}$$

4.1.2.1 Mean Average Precision (MAP) Mean average precision (MAP) is a prominent statistic for recommender systems (Schröder et al. 2011). It determines the precision of the recommendations list with the different size that relates to the ranking of the relevant items such as movies for every relevant item. The arithmetic mean of each of these precisions would then be calculated as given in Eq. (28):

$$AP = \frac{\sum_{r=1}^{N}(P(r) \times rel(r))}{number of relevant items} \tag{28}$$

Following that, the Arithmetic Mean of all users' mean precisions (i.e., available in the test set) is computed to obtain the final MAP that is given below in Eq. (29):

$$MAP = \frac{\sum_{u=1}^{M} AP_U}{M} \tag{29}$$

4.1.2.2 Normalized discounted cumulative gain (NDCG) Normalized Discounted Cumulative Gain (NDCG) is a popular ranking-based evaluation metric for recommender systems. In Discounted Cumulative Gain (DCG), every score corresponding to a relevant item is weighted based on its position. The relevance of the recommendations at the top of the list is given more weight, whereas the significance of the ones at the bottom is given less weight.

Table 2 Details of the dataset

Details of the dataset					
Dataset	No. of users	No. of items	Ratings	Rating scale	Contextual variables
LDos-Comoda	268	4381	2278	1–5	12
DePaul Movie	97	79	2270	1–5	3

Table 3 Types of contextual variables studied from LDos-Comoda and DePaul Movie Dataset along with their meanings

Details of the contextual variables		
Dataset	Contextual Variable	Meaning
LDos-Comoda	Mood	Positive, Neutral, Negative
LDos-Comoda	Time	Morning, Afternoon, Evening, Night
LDos-Comoda	location	Home, Public place, Friend's house
LDos-Comoda	day type	Working day, Weekend, Holiday
DePaul Movie	Companion	Family, Partner, Alone
DePaul Movie	Time	Weekend, Weekday
DePaul Movie	location	Home, Cinema

Similarly, the IDCG is the ideal relevance score of the items from top to bottom in the list of recommendations. The NDCG@K is computed as follows:

$$NDCG_{@K} = \frac{DCG_{@K}}{IDCG_{@K}} where DCG_{@K} = \sum_{i=1}^{K} \frac{G_I}{log_2(i+1)} and IDCG_{@K} = \sum_{i=1}^{k^{ideal}} \frac{G_i{}^{ideal}}{log_2(i+1)} d \quad (30)$$

4.2 Datasets

For context-aware recommendation, there exist many datasets related to different domains such as Tourism, restaurants, travel, and movie applications. However, most of the datasets are very small in size in terms of single-user interactions. Since, for the effective offline training of RL-agent, enough interactions of the users are necessary to capture their interests. As a result, in order to appropriately frame the problem of RL with context-aware recommendations, we employed the DePaul Movie and LDos-Comoda datasets, in which each user has a sufficient number of interactions. Furthermore, we excluded users who had less than 10 interactions with the system. Both of the datasets have several contextual variables in which users provide ratings to the movies in a range of 1–5. A rating of 1 indicates that the item is disliked by users; conversely, ratings of 4 and 5 indicate that the item is much more liked by users. Moreover, the contextual variables include user mood, time, emotions, companion, location, season, day type, weather, physical, social, etc. Among these, we used short-term contextual factors, such as time and companion from the DePaul movie dataset, and time and mood from the LDos-Comoda dataset. The details of the dataset are given in Table 2 and different types of contextual variables with their meaning are given in Table 3

Table 4 Hyperparameters of the Agent

Parameters	Values
Train-Test Division	70/30
Non-overlapping users	Yes
Episodes	1000
ε epsilon	0.6
ε Decay rate	$(\varepsilon - 0.1)/1500$
Actor learning rate	0.001
Critic learning rate	0.001
Actor hidden nodes	128
Critic hidden nodes	128
Embedding dimension	100
The standard deviation for exploration	1.5
Batch size	64
Experience Replays	Prioritized and Random Experience Replays
Discount factor γ	0.9
Actor and Critic weight optimizer	Adam
Actor Update	Soft target network update
Activation function on hidden nodes of actor-critic	ReLu
Tau value	0.001
Max Priority value in *PER*	1.0
α, β and β_{const} in *PER*	0.6, 0.4, 0.0001

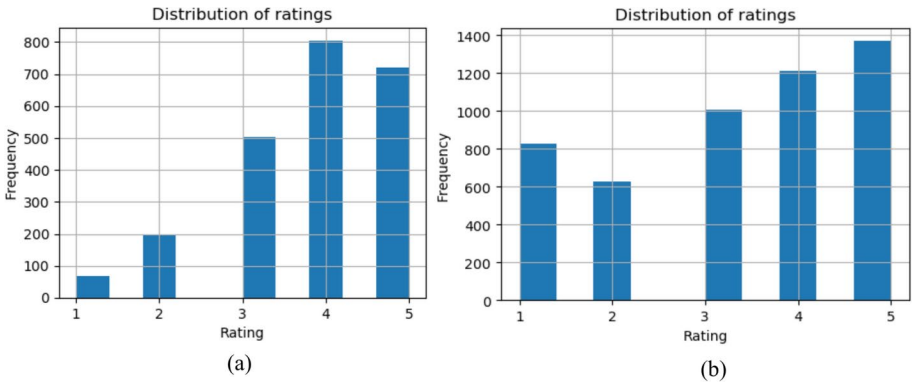


Fig. 6 Rating Distributions of both datasets **a** LDos-Comoda Dataset, **b** DePaul Movie Dataset

and other while the hyperparameters of the model are provided in Table 4. Moreover, the ratings distribution along with frequencies for both datasets are given in Fig. 6.

4.3 Results on LDos-Comoda dataset

In the first stage, we performed the analysis of results on the LDos-Comoda dataset. Initially, the item embedding is learned by an explicit neural collaborative filtering algorithm (E_{NCF}). However, in the original algorithm of NCF, the embedding is learned implicitly

but this learning introduced noise about user preferences since if users rate the item with 1, then still it is considered a positive interaction of the user. Therefore, we convert the training explicitly by directly using the ratings as a label during training with a loss function set to Mean Squared Error (MSE). Following on, we train the RL agent separately for each context and test on the testing set having a different set of users. In order to best evaluate the performance of the recommender agent, we evaluate it with different methods such as with different reward functions such as 5-scale reward (R_c) and binary rewards (R_b). More precisely, in R_c, the agent is awarded within 5-scale ratings i.e., either 1,2,3,4, or 5 while in R_b if the rating is 3 or above, the agent is given a reward of 1, otherwise a 0. Similarly, we also evaluated the algorithm with different experience replays such as simple random experience replays (R) and Prioritized experience Replays (PER). All of these methods are separately evaluated with different contextual variables as well as with different context-modeling strategies suggested in this study i.e., C_SZW and C_SAW. Table 5 shows the results of results on LDos-Comoda dataset with C_SZW_{Rc}, $C_SAW_{Rc,}$ $C_SZW_{Rb,}$ $C_SAW_{Rb,}$ $C_SZW_{PERc,}$ C_SAW_{PERc} $C_SZW_{PERb,}$ C_SAW_{PERb} methods with contextual variable "time". This time variable has possible values morning, afternoon, evening, and night during which users provide the contextual rating. Time is an implicit contextual variable, which implies that it may be collected by the system without human intervention. Table 5 lists the results in terms of precision and recall at different thresholds. It is observed from Table 5 that with "time" context, the highest precison@5 is achieved when the attention weighting (C_SAW) module is used with prioritized experience replays (PER) along with R_b and R_c. Likewise, if we observed the results in terms of experience replays from Table 5, then the precision is high with prioritized experience. Since in PER the instances are sampled from the replay memory according to the temporal difference error.

Subsequently, in Table 6, the results with another contextual variable from the LDosComoda dataset are given i.e., "Mood". In general, the user's "mood" is a type of explicit context because it cannot be easily deduced by the system. The same experimentation is done with this contextual variable and results are computed. It is observed that with "mood" based modeling the highest precsion@5 is achieved with C_SZW of about 0.7277. This means that 73% of the items recommended by the agent in the top 5 recommendation lists are consumed by the users. This demonstrates the recommender agent's strong effectiveness in providing more tailored recommendations based on user mood. Following on, in Tables

Table 5 Results of both attention and zero weighting (C_SZW, C_SAW) with different reward methods (R_c and R_b) and experience replays (R, PER) on LDos-Comoda Dataset with contextual variable "Time"

With the contextual variable "Time"						
Method	Precision@5	Precison@10	Precision@20	Recall@5	Recall@10	Recall@20
C_SZW_{Rc}	0.67	0.66	0.57	0.23	0.50	0.79
C_SZW_{PERc}	0.71	0.71	0.59	0.27	0.56	0.82
C_SZW_{Rb}	0.64	0.68	0.56	0.22	0.51	0.78
C_SZW_{PERb}	0.64	0.58	0.52	0.25	0.47	0.77
Avg_{CsZw}	0.665	0.6575	0.56	0.2425	0.51	0.79
C_SAW_{Rc}	0.71	0.68	0.57	0.27	0.52	0.80
C_SAW_{Rb}	0.67	0.67	0.53	0.26	0.50	0.76
C_SAW_{PERc}	0.733	0.733	0.56	0.25	0.54	0.78
C_SAW_{PERb}	0.733	0.6444	0.5499	0.28	0.46	0.76
Avg_{CsAw}	**0.7115**	**0.68185**	**0.56**	0.265	0.505	0.775

Table 6 Results of both attention and zero weighting (C_SZW, C_SAW) with different reward methods (R_c and R_b) and experience replays (*R, PER*) on LDos-Comoda Dataset with contextual variable "Mood"

With the contextual variable "Mood"						
Method	Precision@5	Precison@10	Precision@20	Recall@5	Recall@10	Recall@20
C_SZW_{Rc}	0.711	0.6555	0.56666	0.2427	0.5018	0.78417
C_SZW_{PERc}	0.755555	0.7	0.59444	0.2872	0.529496	0.8237
C_SZW_{PERb}	0.6666	0.6777	0.5722	0.22427	0.502324	0.79614
C_SZW_{Rb}	0.7777	0.711	0.6000	0.318	0.54645	0.8255
$Mean_{CsZw}$	**0.7277**	**0.686**	**0.5833**	0.2680	0.520018	0.8073
C_SAW_{Rc}	0.733	0.7	0.577	0.28	0.52	0.79
C_SAW_{Rb}	0.622	0.67777	0.55	0.22274	0.5560	0.7831
C_SAW_{PERc}	0.6444444	0.6555	0.5611	0.25261	0.5017	0.79044
C_SAW_{PERb}	0.68888	0.677777	0.561111	0.231	0.5222	0.785888
$Mean_{CsAw}$	0.6720	0.67762	0.5623	0.2465	0.5249	0.787

Table 7 Results of both attention and zero weighting (C_SZW, C_SAW) with different reward methods (R_c and R_b) and experience replays (*R, PER*) on LDos-Comoda Dataset with contextual variable "day type"

With the contextual variable "Daytype"						
Method	Precision@5	Precison@10	Precision@20	Recall@5	Recall@10	Recall@20
C_SZW_{Rc}	0.6666	0.7111	0.6000	0.227	0.529	0.8204
C_SZW_{PERc}	0.733	0.7000	0.5666	0.2700	0.522	0.7947
C_SZW_{Rb}	0.711	0.644	0.566	0.2724	0.4872	0.778
C_SZW_{PERb}	0.6888	0.655	0.5777	0.2601	0.485	0.792
$Mean_{CsZw}$	**0.69985**	**0.67775**	**0.57775**	0.25733	0.5058	0.796275
C_SAW_{Rc}	0.7555	0.6666	0.56111	0.2865	0.5145	0.7884
C_SAW_{Rb}	0.644	0.688	0.5944	0.2147	0.51315	0.826
C_SAW_{PERc}	0.644	0.677	0.583	0.221	0.5053	0.7998
C_SAW_{PERb}	0.75	0.7	0.55	0.284	0.578	0.775
$Mean_{CsAw}$	**0.698375**	**0.6829**	**0.5721**	0.25155	0.52775	0.7973

7 and 8, the results with contextual variable "Day type" and "Location" is given. With the "Day type" context, the results of both C_SZW and C_SAW methods in terms of precision@5 are the same i.e., 0.699985 and 0.698375.

However, with some contextual variables, the results are not higher with (*PER*). This is due to the reason of insufficient coverage of the samples by (*PER*) since it favors the small subset of state spaces (Pan, et al. 2022). From all of these results, it is observed that contextual modeling with C_SZW performs better than C_SAW. The C_SZW focuses on the user's most recent context while modeling their interest and preferences and assigns zero weighting to the non-relevant items.

In addition, the experimentation on different sizes of states (i.e., user previous positive interaction history) is also studied whose results are depicted in Fig. 6. More precisely, Fig. 7 shows the separate results for each contextual variable including "day type", "location", "time" and "mood". It is found that state size has a substantial relationship with recommender agent performance. Since users' large past interaction histories (i.e., state size) add noise concerning their current interests, hence, the more recent fewer items liked by users are more valuable. As a result, by increasing the size of the state, the performance declines.

Table 8 Results of both attention and zero weighting (C_SZW, C_SAW) with different reward methods (R_c and R_b) and experience replays (R, PER) on LDos-Comoda Dataset with contextual variable "location"

With the contextual variable "Location"						
Method	Precision@5	Precison@10	Precision@20	Recall@5	Recall@10	Recall@20
C_SZW_{Rc}	0.6444	0.6444	0.56111	0.2518	0.5020	0.7975
C_SZW_{PERc}	0.755	0.680	0.5722	0.3123	0.5410	0.78500
C_SZW_{Rb}	0.733	0.688	0.6055	0.281	0.4909	0.8278
C_SZW_{PERb}	0.755	0.6222	0.566	0.2826	0.4789	0.7917
$Mean_{CsZw}$	**0.72185**	**0.6585**	**0.5762**	0.2819	0.5032	0.8005
C_SAW_{Rc}	0.666	0.655	0.5611	0.2554	0.466	0.784
C_SAW_{Rb}	0.711	0.700	0.5777	0.25522	0.5264	0.7958
C_SAW_{PERc}	0.733	0.688	0.5833	0.2502	0.511	0.8078
C_SAW_{PERb}	0.666	0.666	0.56111	0.2584	0.5020	0.7843
$Mean_{CsAw}$	0.694	0.6775	0.570803	0.254805	0.50135	0.7925

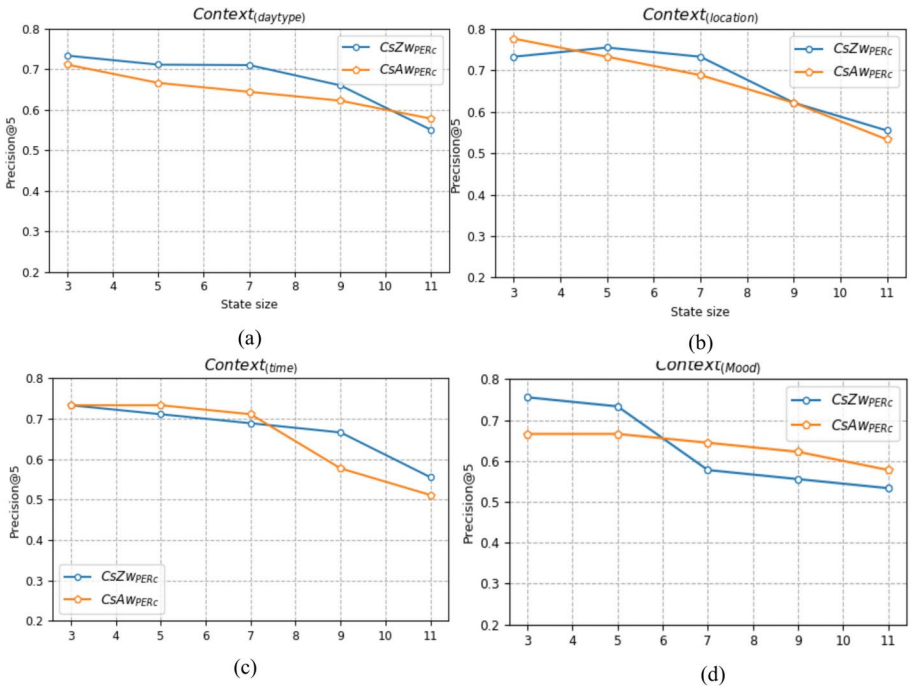


Fig. 7 Parameter Study of state size vs precicion@5 in both methods of context-modeling with different contextual variables on LDos-Comoda Dataset (**a**) With contextual variable day type (**b**) With contextual variable location (**c**) With contextual variable time (**d**) with contextual variable mood

Following on, we further perform the experiments by lowering the decay rate value of ε epsilon. Table 16 in Appendix I shows the findings, and it is evident that lowering the rate of interaction reduces the outcomes in most scenarios since the agent becomes more "adventurous" in this case, i.e. the agent investigates more about the user's interests and preferences, and too much exploration reduces the performance. Similarly, a parameter analysis of interaction length vs agent performance in terms of precision@5 is also undertaken,

the results of which are shown in Fig. 8. Figure 8 shows that large interaction sizes during training lead to a decrease in performance owing to excessive exploration of user interests and preferences. The graphs show that precision@5 initially increases and subsequently decreases by increasing the length of interaction.

4.4 Results on DePaul Movie dataset

In the second stage, we evaluated the proposed model on the Depaul movie dataset on different time and companion-based contextual variables. Similar to the LDos-Comoda dataset, we have trained the item embedding for this dataset using explicitly designed NCF (E_{NCF}). The results of the Depaul movie dataset in terms of precision and recall with different topK thresholds are given in Tables 9, 10, and 11. The chosen contextual variables of this dataset include the "companion", "time", and "location". The results with each contextual variable are separately listed in Tables. If we analyze the results of the "companion" contextual variable, then the highest precision@5 is about 0.5394 achieved with C_SZW. Similarly, the results of "time" and "location" contextual variables are given in Tables 10 and 11. Both of these are implicit contextual variables as both time and location are directly inferred by the system. However, the "companion" of the user is challenging to be directly inferred until the users explicitly provide them. The highest precision@5 is observed with the C_SZW

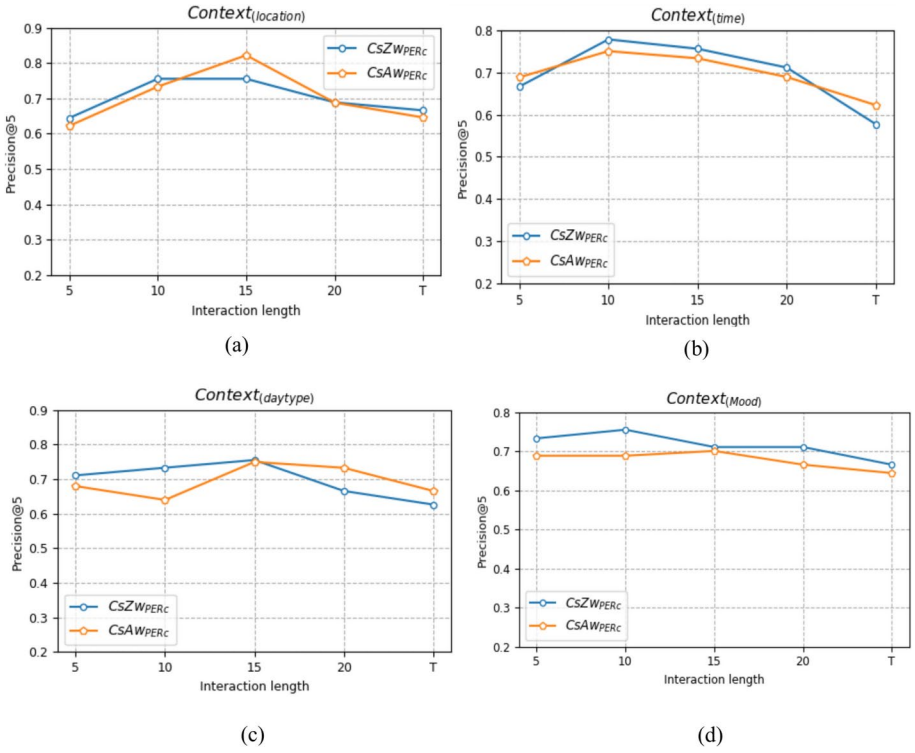


Fig. 8 Parameter Study of interaction length vs presicion@5 in both methods of context-modeling with different contextual variables on LDos-Comoda Dataset (**a**) With contextual variable location (**b**) With contextual variable time (**c**) With contextual variable day type (**d**) with contextual variable mood

Table 9 Results of both attention and zero weighting (C_SZW, C_SAW) with different reward methods (R_c and R_b) and experience replays (*R, PER*) on the DePaul Movie dataset with contextual variable "Companion"

With the contextual variable "Companion"						
Method	Precision@5	Precison@10	Precision@20	Recall@5	Recall@10	Recall@20
C_SZW_{Rc}	0.5684	0.4421	0.3000	0.5875	0.8120	0.9718
C_SZW_{PERc}	0.5368	0.4473	0.307	0.5221	0.81995	0.980
C_SZW_{Rb}	0.5052	0.43684	0.3052	0.4882	0.8074	0.978
C_SZW_{PERb}	0.5473	0.46315	0.3000	0.5340	0.85124	0.97055
$Mean_{CsZw}$	**0.5394**	**0.447348**	**0.3030**	0.5329	0.822648	0.975
C_SAW_{Rb}	0.5157	0.42105	0.3000	0.53006	0.7892	0.9705
C_SAW_{Rc}	0.5578	0.4684	0.29736	0.5419	0.83409	0.9647
C_SAW_{PERc}	0.473	0.41578	0.30263	0.401	0.7647	0.97446
C_SAW_{PERb}	0.6105	0.4789	0.30789	0.60496	0.8571	0.982
$Mean_{CsAw}$	0.5392	0.4460	0.30197	0.51948	0.8112	0.9729

Table 10 Results of both attention and zero weighting (C_SZW, C_SAW) with different reward methods (R_c and R_b) and experience replays (*R, PER*) on DePaul Movie Dataset with contextual variable "Time"

With the contextual variable "Time"						
Method	Precision@5	Precison@10	Precision@20	Recall@5	Recall@10	Recall@20
C_SZW_{Rc}	0.51578	0.44210	0.3052	0.4965	0.790	0.9786
C_SZW_{PERc}	0.4842	0.4421	0.29736	0.40305	0.7970	0.967418
C_SZW_{Rb}	0.5368	0.452631	0.2947	0.46091	0.78899	0.96412
C_SZW_{PERb}	0.5368	0.48421	0.292	0.533	0.84648	0.9597
$Mean_{CsZw}$	**0.51839**	**0.45526**	**0.2973**	0.4733	0.80516	0.96746
C_SAW_{Rb}	0.5473	0.43684	0.2947	0.5791	0.81421	0.96303
C_SAW_{Rc}	0.4421	0.4263	0.2947	0.3357	0.8034	0.9647
C_SAW_{PERc}	0.431578	0.4052	0.2947	0.38796	0.7347	0.96130
C_SAW_{PERb}	0.494736	0.4263	0.3	0.52510	0.766896	0.97164
$Mean_{CsAw}$	0.4789	0.42366	0.2906	0.4569	0.779802	0.96516

Table 11 Results of both attention and zero weighting (C_SZW, C_SAW) with different reward methods (R_c and R_b) and experience replays (*R, PER*) on DePaul Movie Dataset with contextual variable "Location"

With the contextual variable "Location"						
Method	Precision@5	Precison@10	Precision@20	Recall@5	Recall@10	Recall@20
C_SZW_{Rc}	0.4210	0.4052	0.2947	0.4233	0.768	0.964
C_SZW_{PERc}	0.4736	0.4210	0.286	0.4634	0.78013	0.9509
C_SZW_{Rb}	0.5578	0.452	0.2973	0.56772	0.8366	0.96726
C_SZW_{PERb}	0.4315	0.37894	0.2947	0.40856	0.7324	0.96256
$Mean_{CsZw}$	0.4709	0.414	0.293175	0.4657	0.779	0.96118
C_SAW_{Rb}	0.4736	0.4368	0.2894	0.4452	0.7860	0.955
C_SAW_{Rc}	0.5052	0.4368	0.3000	0.4889	0.8114	0.9705
C_SAW_{PERc}	0.4736	0.415	0.2868	0.36687	0.771	0.9505
C_SAW_{PERb}	0.484	0.4210	0.2973	0.5161	0.79909	0.96632
$Mean_{CsAw}$	**0.4841**	**0.4274**	**0.2933**	0.454	0.7918	0.9605

module, and with the time contextual variable of the Depaul movie while with the location has the highest precison@5 is about 0.4841 with the C_SAW module. Similar to the previous dataset, the C_SZW-based context modeling performs well in comparison with C_SAW. Because the C_SZW emphasizes the user's most recent context while modeling their interests and preferences, non-relevant items are assigned zero weighting. However, if the results of both datasets are compared then the performance of the recommender agent is less with the DePaul movie dataset.

This is because the DePaul move dataset has a small number of users, and their interactions with the system are also limited. As a result of having so few interactions, the agent has difficulty learning the user's interests and preferences. As a result, less precise agent training will yield fewer results. Furthermore, as we did for the LDos-Comoda dataset, we examined the impact of different sizes of states (i.e. user past positive interaction history), the results of which are shown in Fig. 9. Similar to the previous dataset, we have performed the experimentation by different interaction lengths during training as shown in Fig. 10.

It is observed that the results decline when the size of the user's interaction increases to large values, and this indicates that the agent will explore more about users' interests. Moreover, large exploration will introduce noisy learning about user's interests and preferences. Furthermore, an attempt is also made to improve the performance of the agent on the DePaul movie dataset by decreasing the rate of decay of epsilon. The results of this

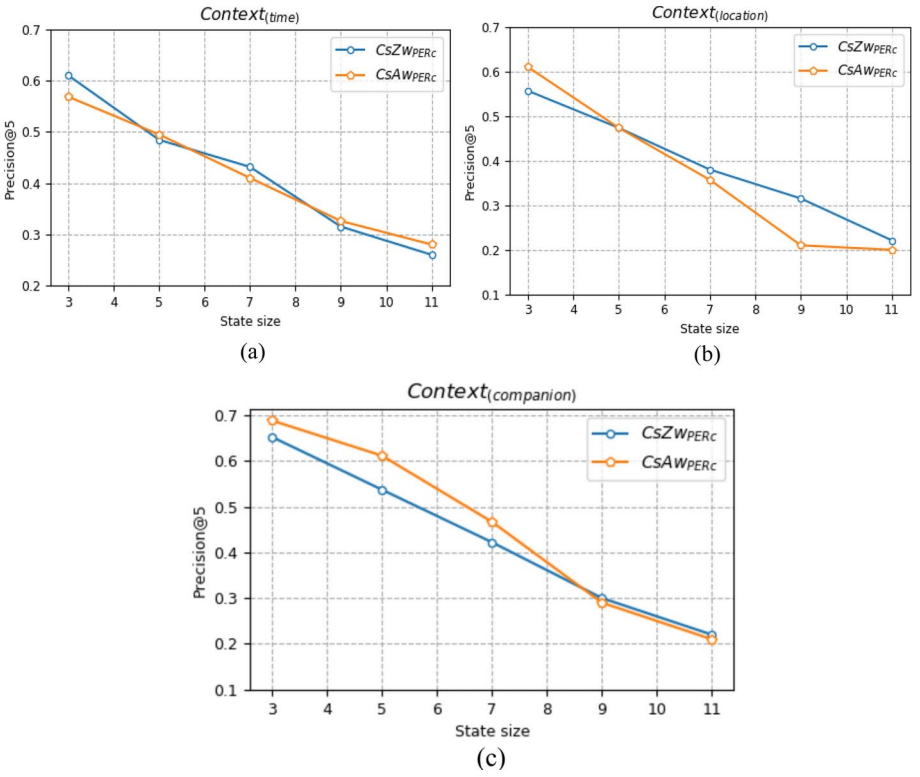


Fig. 9 Parameter Study of state size vs precicion@5 in both methods of context-modeling with different contextual variables on DePaul Movie Dataset (**a**) With contextual variable time (**b**) With contextual variable location (**c**) With contextual variable companion

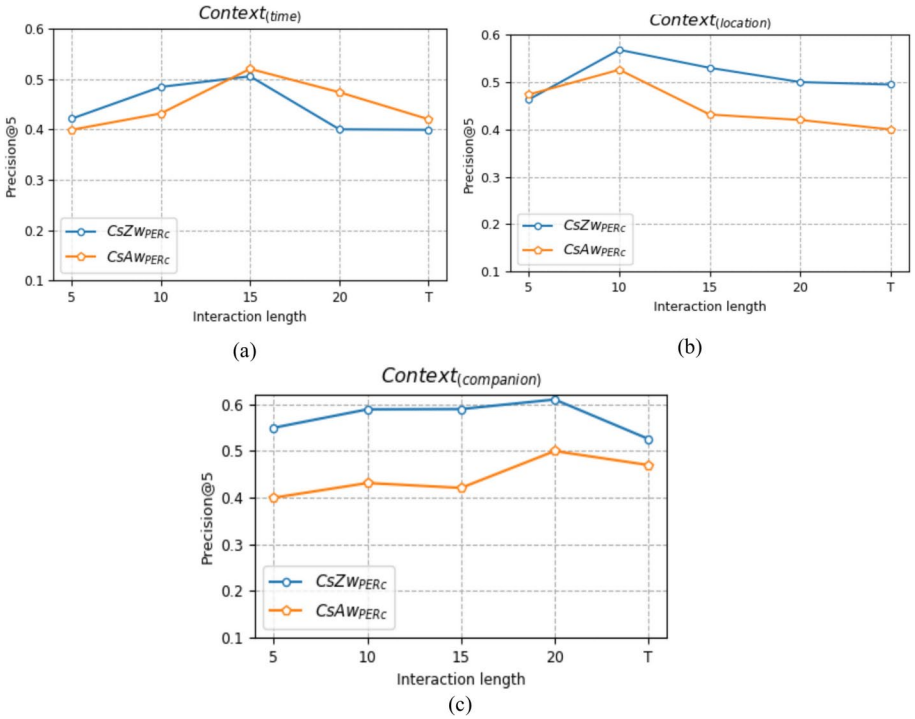


Fig. 10 Parameter Study of Interaction length vs precicion@5 in both methods of context-modeling with different contextual variables on DePaul Movie Dataset (**a**) With contextual variable time (**b**) With contextual variable location (**c**) With contextual variable companion

experiment are given in Table 17 in Appendix I. It is observed that the performance of the agent improves as we lower the decay rate since in the DePaul movie dataset the rating distribution of items for value 1 or 2 is more as in comparison with LDos-Comoda. As a result, an agent requires a little large exploration and slowly progresses to exploitation to find the optimal items for users with ratings of 4 or 5.

4.5 *Results of item embedding with* E_{NCF}

In most of the existing studies of RL-based recommender systems, the results of item embedding's that are learned by their specific methods are not reported. However, they are hidden performance metrics since they are part of actor and critic states or any other RL-based agent states. In this work, the results of the explicitly trained NCF model are also shown in Table 12. The results of train and test MSE on the LDos-Comoda dataset are about 0.17 and 0.023, while on the DePaul movie dataset, it is 0.06 and 0.0052 respectively. More explicitly, the NCF models are separate embedding layers for users and items to encode the sparse data into dense vector representations. Followed by these layers, two hidden layers are added having units 128 and 64. Subsequently, the last layer has 1 unit to forecast the user interest and preference in terms of ratings. During training, the negative and positive sample ratio size is 0.8 with batch size set to 128. The model is trained for 200 epochs and later on, evaluated on a test set comprising the data of users different from those present in train-

Table 12 Result of explicitly trained NCF E_{NCF} on Depaul movie and LDos-Comoda dataset

Dataset	Train (MSE)	Test (MSE)	Embedding dimension	Model Architecture
LDos-Comoda	0.17308	0.023893	100	Same
DePaul Movie	0.0647757	0.005213	100	Same
Average	0.118927	0.014553	–	–

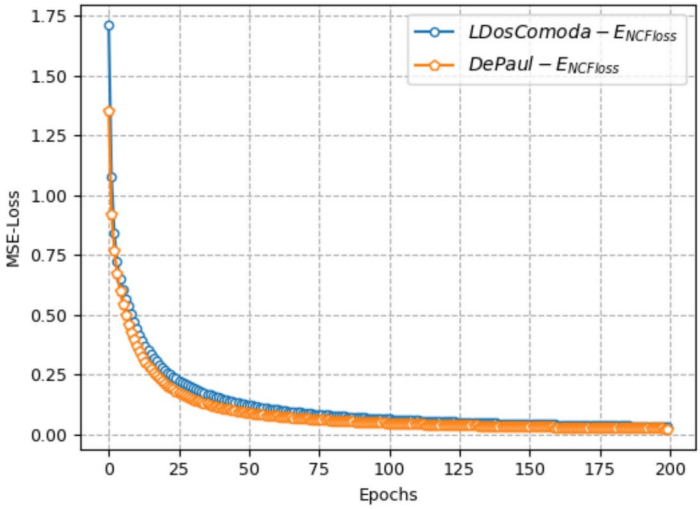


Fig. 11 Loss curves of E_{NCF} on Depaul movie and LDos-Comoda dataset

ing. Moreover, in E_{NCF}, the training loss function of the agent is based on regression loss namely the "MSE" (mean squared error), since this study treats the actual feedback in terms of ratings as a label sample. In each epoch, the batch of instances are sampled to train E_{NCF} from which some of them are rated by users and others for which feedback is not provided. During training, the label 0 is given to samples for which feedback is not provided. The loss curves of both the LDos-Comoda and DePaul movie datasets during training are depicted in Fig. 11. It is observed that the loss values converge smoothly as the epoch increases.

4.6 Performance of list-wise context-aware recommender agent

In this study, the list-wise context-aware recommender agent is also suggested to evaluate the performance and application of the proposed work. More precisely, in the list wise recommender system, the agent is allowed to recommend the list of items to users depending upon the context of the users. The list-wise agent observes the user's positive interaction history at time step T as well as the context of the users. Later on, the user's current contextual state items, rank the list of items I to be recommended to the user at the next time step $T+1$. In this case, the feedback of the agent is either the sum or mean reward of the feedback (ratings) of the items present in the list. Based on this feedback, the agent either adds or ignores the list of items recommended to the user's current state to perform the state transition. For such an experimental setup, we train the agent with different contextual

variables on both datasets separately and compute the precision and recall values with different topK thresholds. In addition, the experimentation is also performed with different reward methods such as 5-scale reward (R_c) and binary rewards (R_b) as well as simple random experience replays (R) and Prioritized experience replays (PER). These methods are $L\text{-}C_SZW_{PERb}$, $L\text{-}C_SZW_{PERbavg}$ $L\text{-}C_SAW_{PERb,}$ $L\text{-}C_SAW_{PERbavg,}$ $L\text{-}C_SZW_{PERc}$ $L\text{-}C_SZW_{PERcavg}$ $L\text{-}C_SAW_{PERc}$ $L\text{-}C_SAW_{PERcavg}$ $L\text{-}C_SZW_{Rb,}$ $L\text{-}C_SZW_{Rbavg}$ $L\text{-}C_SAW_{Rb}$ $L\text{-}C_SAW_{Rbavg}$ $L\text{-}C_SZW_{Rc}$ $L\text{-}C_SZW_{Rcavg}$ $L\text{-}C_SAW_{Rc}$ and $L\text{-}C_SAW_{Rcavg.}$ The results of the LDos-Comoda Dataset on different contextual variables including "time", "mood", "location", and "day type" is given in Tables 3,4, 5, and 6 while the results of list-wise testing on the DePaul movie dataset on "Location", "Time", and "Companion" are given in Tables 7, 8, and 9 respectively. It is observed from the results that with the list-wise context-aware strategy of recommendation, the agent recommender performs better than the single-item strategy. It is because an item from a list of items is likely to be selected by the user, and an agent receives cumulative feedback on a set of items to become more familiar with the user's interests and preferences. Hence, adopting the list-wise strategy on context-aware recommender systems will become more advantageous than the single-item strategy. As it is evident from the results, with "time" context in the LDos-Comoda dataset, the highest precison@5 is about 0.86. Similarly, with the contextual variable "mood", it is about 0.911 while with "location" and "day type" it is 0.88 and 0.911 respectively. Similarly, in the DePaul movie the precision@5 with "location", "time", and "companion" are 0.66, 0.64, and 0.65 respectively.

4.7 Analysis, discussions and comparisons

Context-aware recommender systems are now considered a novel subject, with little research undertaken in this area (Lam et al. 2021; Chen and Xia 2021; Li et al. 2021). Previous methods and strategies cannot satisfy the requirements and interests of users in the current context. In addition, some traditional methods are designed but they view the recommendation as static and do not adapt themselves by changing the interests of users over time in a different context. As a result, we built the context-aware recommender utilizing RL, which views recommendation as a dynamic procedure, and we improved it further by combining the idea of user context with such a dynamic way of recommendation. It is observed from the above-mentioned results that the proposed model performs well in capturing users' recent contextual interests while making a recommendation. We have analyzed the performance of the proposed model with different contextual variables in terms of top 5, 10, and 20 precision and recalls. Furthermore, the influence of various contextual factors on precision and recalls is investigated independently. More precisely, the first context-specific modeling module is based on a zero-weighting strategy. In this strategy, historical items that are not relevant to the user's present context are assigned zero weights, and filtering is undertaken on those items that are relevant to the current context of the user. Similarly, in the second strategy, instead of completely assigning those items with zero weighting, we have assigned the weights to all recently clicked items of users by using the dot-production layer. In this layer, scores are assigned depending on the similarity between the query and value embedding. The query embeddings are formed up of items that are relevant to the user's most recent context; however, the value embedding is made up of items that are not related to the user's most recent context but come underutilizing recently clicked items.

Table 13 Comparison of the proposed model with "without context recommender agent" on LDos-Comoda Dataset

Method	Dataset	Context	Precison@5	Precison@10	MAP@10	NDCG@10
Without Context$_{PERc}$	LDos-Comoda	No	0.67	0.64	0.62	0.62
C_SZW_{PERc}	LDos-Comoda	mood	**0.725**	**0.6625**	**0.72**	**0.69**
C_SZW_{PERc}	LDos-Comoda	time	**0.77**	**0.71**	**0.73**	**0.72**
C_SZW_{PERc}	LDos-Comoda	daytype	0.66	0.64	**0.66**	**0.72**
C_SZW_{PERc}	LDos-Comoda	location	0.66	0.60	**0.68**	**0.72**
C_SAW_{PERc}	LDos-Comoda	mood	**0.73**	**0.67**	**0.68**	**0.68**
C_SAW_{PERc}	LDos-Comoda	time	**0.70**	**0.70**	**0.76**	**0.74**
C_SAW_{PERc}	LDos-Comoda	day type	0.64	0.63	**0.645**	**0.70**
C_SAW_{PERc}	LDos-Comoda	location	**0.73**	**0.66**	**0.69**	**0.74**

Table 14 Comparison of the proposed model with "without context recommender agent" on DePaul Movie Dataset

Method	Dataset	Context	Precison@5	Precison@10	MAP@10	NDCG@10
Without Context$_{PERc}$	DePaul Movie	No	0.57	0.45	0.60	0.58
C_SZW_{PERc}	DePaul Movie	time	**0.62**	**0.47**	**0.62**	**0.59**
C_SZW_{PERc}	DePaul Movie	location	**0.68**	**0.46**	**0.67**	**0.61**
C_SZW_{PERc}	DePaul Movie	Companion	**0.61**	**0.50**	**0.61**	**0.60**
C_SAW_{PERc}	DePaul Movie	time	**0.60**	**0.46**	0.60	0.58
C_SAW_{PERc}	DePaul Movie	location	**0.68**	**0.47**	**0.65**	**0.61**
C_SAW_{PERc}	DePaul Movie	Companion	**0.66**	**0.48**	**0.65**	**0.62**

The suggested solutions would provide dynamic states, which is beneficial in cold start situations where the user does not have enough interactions. In addition, the zero weighting performs well since it models the user's most specific interest, however, this module does not ensure diversity in recommendation lists. However, in order to accommodate the diversity of recommendations, rather than selecting only specific items, the attention module assigns tiny weights rather than simply ignoring or zero. This is an excellent method, for example, if the user's mood is sad, recommending similar items to the user that they enjoy most in a happy mood would convert their sad mood to a happy mood, leading to user pleasure and excitement when interacting with systems. So, diversification is a key component to consider when making recommendations so that users are not bored with the same items over and again. However, there exists a trade-off between diversification and accuracy. To further indicate the performance of the proposed model, we have performed a comparison with "without a context RL recommender agent" with the proposed context-specific recommender agent. In "without" context-based recommender agent, we train and test the actor-critic model without any context modeling (i.e. Section 3.2.2). More precisely, in this, the states simply consisted of recent item embedding's learnt through E_{NCF}. The comparison results in terms of overall MAP@10, NDCG@10, Precision@5, and Precision@10 are given in Tables 13 and 14. More precisely, the first two rows in Tables 13 and 14 shows results of recommender system "without context". It is observed on the LDos-Comoda dataset that precision@5, precision@10, MAP@10, and NDCG@10 are increased with context "mood", "time", and "location" in comparison with without contexts. However, with "day type" context, the MAP@10 and NDCG@10 are also increased in comparison with those

without context with both attention and zero weighting strategy. The improved results are indicated in bold values.

Similarly, in the DePaul movie dataset, all contextual factors increase performance when compared to without context RL-agent. All of these experiments are done with parameter setting i.e., (PER_c) for a fair comparison. This comparison shows that the user's context has a significant influence on their present interests and preferences. Furthermore, the suggested RL agent examines not only the context for modeling user preferences but also the item selected by the user in the most recent context. Furthermore, we may conclude which context factor is most significant to the model based on the highest findings of distinct contextual variables.

Moreover, the comparison of the proposed work with non-reinforcement methods available in DeepCARSKit (Zheng 2022b) is shown in Fig. 12. More explicitly, the first graph depicts the comparison in terms of MAP@10 while the second graph shows the comparison in terms of precision@5 as well as precision@10. It is observed from Fig. 12 that the results of the proposed reinforcement learning model outperform in comparison with context-based variants of neural collaborative filtering algorithms and factorization machine and deep factorization machine, especially in terms of precision@5 and precision@10. However, there is around a 1% improvement in terms of MAP@10. These comparable and enhanced results demonstrate our method's potential. Furthermore, instead of focusing just on score level increase, another advantage of the proposed work is its ability to adapt to changing user preferences rather than seeing recommendations as a prediction problem.

In addition to the above, we have also performed a generic comparison with existing studies in which RL-based agents are designed to generate recommendations. However, we did not perform a scoring comparison because they were using different datasets, but we used context-aware datasets. Due to differences in sparsity and user interactions in both types of datasets, comparing them in terms of scores is not fair. As a result, Table 15 highlights a general and summarized comparison highlighting the novelty and contribution of this work to the literature. More specifically, there are several RL-based research in which efficient recommender agents have been built; however, none of these include any study in which the context of the user is considered while modeling states and making recommendations. In addition, most of the studies have employed the actor-critic model, since the deep-Q-learning methods have large temporal complexity.

This is owing to the fact that they calculate the Q-value for each action. Similarly, the popular evaluation metrics that were utilized for evaluating the recommender agent are

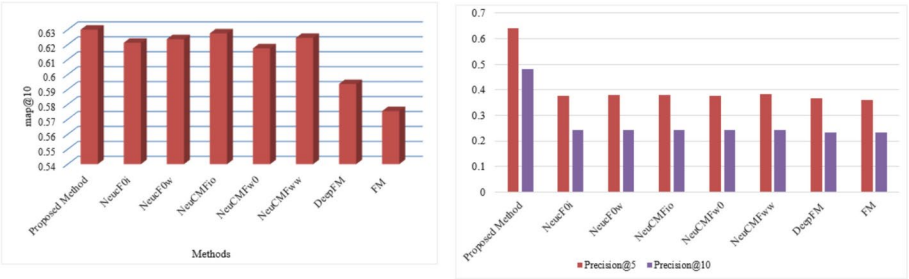


Fig. 12 Comparative analysis of proposed work with state-of-the-art DeepCARSKit (Zheng 2022b) algorithms

Table 15 Comparison of the proposed model with existing RL studies for recommendations

Existing Studies	RL-Model	User-Context	Datasets	List-wise	Additional feature in States Modeling	Assessment Metric
Liu et al. (2020)	Actor-Critic	No	MovieLens, Yahoo Music, Jester	No	Demographic and Items Info	MAP, NDCG
Zhao (2021)	Actor-Critic	No	MovieLens	No	Users, tags, movies, genres	RMSE, MAE
Zhao, et al. (2018b)	Actor-Critic	No	E-commerce dataset	Yes	Clicked Items	Precision, recall, NDCG, MAP
Liu et al. (2106)	Actor-Critic	No	MovieLens	No	Clicked Items of group	Recall and NDCG
Hu et al. (2017)	Q-learning	No	Music dataset	No	Recently listen songs	Accuracy, Coverage, Score_list
Lei and Li (2019)	Q-Network	No	Movie Lens	No	Rating vectors	Average ratings
Yuyan et al. (2019)	Q-Network	No	MovieLens	No	Users and movie features (age, job, genre, occupation, etc.)	RMSE
Zhao, et al. (2018a)	DEERs Framework	No	JD.com dataset	No	Clicked Items/ Skipped Items	MAP and NDCG
Liu, et al. (2021)	Actor-critic	No	MovieLens	No	Recent Clicked Items	Precision and Diversity
Ours	Actor-Critic	Yes	LDos-Comoda, DePaul	Yes	Items with Contextual Variables	Precision, Recall, MAP, NDCG

the ranking metrics such as precision, recall, MAP, NDCG, and AP, However, some have utilized the RMSE and MAE because in their recommender agent, the actions are directly considered as ratings. The objective of the agent is to forecast the ratings on an item and later on the recommendations are provided based on ratings. In addition, most of the studies considered the one-item recommendation at each time step, however, Zhao et al. (2018b) have proposed the concept of list-wise recommendation. More precisely, in their method, a list of items is recommended to the user at a specific time step.

It is concluded and evident from Table 15 that the work presented in this study introduces a new idea of modeling states of RL-agent with user-specific context since the context has a great impact on rating behavior. Furthermore, we have made an effort to introduce some diversity because of user context by including an attention layer in which attention scores or weights are allocated to recently clicked items (Liu, et al. 2021). One of the proposed method's strengths is that it produces recommendations while considering their context, and it is adaptable to user-evolving interests because of RL. Furthermore, in this study, a list-wise variant is also suggested, in which a list-wise of user context-aware items is recommended. Because of the consideration of user context, the suggested algorithm generated more personalized recommendations.

In addition, it is also necessary to acknowledge the limitation of the study, therefore, one possible limitation is the data. More specifically, context-based datasets are small in terms of the number of users, sparsity, and interactions, making it difficult to effectively train the

RL agent. In the future, we will create our datasets for context-aware recommendations to tackle the problem of context-aware recommendation using RL more effectively. Furthermore, the limited data problem can be addressed if user groups are formed based on trust links; this will ultimately solve the problem of fewer interactions with the system because the group contains a large number of users, and combining their interactions will result in more interaction than looking at single user-interaction. Moreover, Although our context-based reinforcement recommender system performs well in personalized recommendation tasks, but besides that, we have also acknowledge the limitations of reinforcement learning regarding its longer training times as well as requires a huge amount of interactions data (Ding and Dong 2020; Yu 2018). However, the latest research studies attempt to overcome such limitations RL in domain of recommender systems such as self-supervised learning for RL-recommender systems (Xin, et al. 2020), apprenticeship learning via IRL algorithm as well as addressing interpretability issues utilizing a user-centric path reasoning framework (Tai, et al. 2021; Lin 2023). To solve such challenges and improve this work, future research should look at novel approaches like transfer learning from related domains with additional sources of data or exploiting auxiliary information like user preferences acquired from social media platforms.

5 Conclusion

Recommender systems or agents are one systematic solution in the present era of big data, a massive volume of information in practically every domain. Furthermore, user interests and preferences in different contexts may fluctuate and evolve. Traditional recommended tools are not sufficient to model such evolving interest in different user contexts. Therefore, in this paper, we proposed an RL-based context-aware recommender agent that models the user's interests and preferences depending upon the contextual environment they have during interactions with the system. We proposed two distinct unique techniques that precisely filter the user's present interest while taking into account the user's context, such as mood, time, companion, etc. An extensive set of experiments is performed over the popular context-aware large movie datasets to indicate the performance method. More precisely, the highest MAP@10 on the LDos-Comoda dataset is about 76% with involving users-context "time", similarly, the highest MAP@10 on the DePaul Movie dataset is about 67% with contextual variable "location". It is also observed that the proposed method shows good results in comparison with "without context recommender models" with a mean improvement of 8% on the LDos-Comoda dataset and 3% on the Depaul-Movie dataset. These findings suggest that the user's context is a significant aspect of generating personalized recommendations. In addition, his work also stimulates researchers to develop context-aware recommendation models with more optimal ways for incorporating the user's context into deep reinforcement learning models. In the future, while providing recommendations, we will enrich the proposed model using user trust data.

Table 16 Results of both attention and zero weighting (C_SZW, C_SAW) by lowering the decay rates of ε epsilon on the LDos-Comoda dataset

Method	Precision@5	Precison@10	Precision@20	Recall@5	Recall@10	Recall@20
With the contextual variable "mood"						
C_SZW_{PERc}	0.72500	0.6625	0.66	0.19	0.35	0.735
C_SAW_{PERc}	0.7300	0.66	0.50	0.40	0.64	0.84
With the contextual variable "time"						
C_SZW_{PERc}	0.775	0.7125	0.70	0.23	0.41	0.76
C_SAW_{PERc}	0.70	0.70	0.6625	0.22	0.42	0.734
With the contextual variable "location"						
C_SZW_{PERc}	0.66	0.60	0.49	0.39	0.61	0.833
C_SAW_{PERc}	0.73	0.67	0.51	0.40	0.64	0.85
With contextual variables "day type"						
C_SZW_{PERc}	0.66	0.64	0.50	0.37	0.64	0.84
C_SAW_{PERc}	0.64	0.63	0.49	0.37	0.63	0.84
Mean (C_SZW_{PERc})	**0.705**	**0.6537**	**0.5875**	0.3025	0.5025	0.792
Mean(C_SAW_{PERc})	0.70	0.665	0.5406	0.3475	0.5825	0.82

Table 17 Results of both attention and zero weighting (C_SZW, C_SAW) by lowering the decay rates of ε epsilon on the DePaul Movie Dataset

Method	Precision@5	Precison@10	Precision@20	Recall@5	Recall@10	Recall@20
	With the contextual variable "companion"					
C_SZW_{PERc}	0.61	0.50	0.30	0.62	0.88	0.98
C_SAW_{PERc}	0.66	0.48	0.30	0.67	0.88	0.98
	With the contextual variable "time"					
C_SZW_{PERc}	0.62	0.47	0.29	0.64	0.86	0.97
C_SAW_{PERc}	0.60	0.46	0.31	0.63	0.81	0.985
	With the contextual variable "location"					
C_SZW_{PERc}	0.69	0.46	0.29	0.69	0.85	0.97
C_SAW_{PERc}	0.684	0.47	0.297	0.71	0.87	0.974
Mean (C_SZW_{PERc})	**0.64**	**0.47**	**0.2933**	0.65	0.8633	0.97566
Mean (C_SAW_{PERc})	**0.65**	**0.46**	**0.300**	0.67	0.8533	0.97966

Table 18 List-wise results of both attention and zero weighting ($L\text{-}C_SZW$, $L\text{-}C_SAW$) with different reward methods (R_c and R_b) in both sum and average case ($_{\text{Avg}}$) and experience replays (R, PER) on LDos-Comoda Dataset with contextual variable "Time"

Method	Precision@5	Precison@10	Precision@20	Recall@5	Recall@10	Recall@20
$L\text{-}C_SZW_{PERb}$	0.84	0.64	0.43	0.679	0.88	1.0
$L\text{-}C_SZW_{PERbavg}$	0.82	0.63	0.42	0.67	0.88	1.0
$L\text{-}C_SAW_{PERb}$	0.86	0.67	0.42	0.66	0.90	1.0
$L\text{-}C_SAW_{PERbavg}$	**0.86**	**0.68**	**0.42**	0.66	0.90	1.0
$L\text{-}C_SZW_{PERc}$	0.73	0.54	0.36	0.68	0.88	0.97
$L\text{-}C_SZW_{PERcavg}$	0.733	0.55	0.36	0.66	0.88	0.97
$L\text{-}C_SAW_{PERc}$	0.69	0.57	0.35	0.65	0.90	0.97
$L\text{-}C_SAW_{PERcavg}$	0.71	0.58	0.37	0.65	0.90	0.97
$L\text{-}C_SZW_{Rb}$	0.84	0.65	0.43	0.66	0.88	0.97
$L\text{-}C_SZW_{Rbavg}$	0.84	0.65	0.43	0.66	0.88	0.98
$L\text{-}C_SAW_{Rb}$	0.86	0.65	0.422	0.6826	0.89	0.99
$L\text{-}C_SAW_{Rbavg}$	0.86	0.65	0.422	0.6826	0.89	0.99
$L\text{-}C_SZW_{Rc}$	0.755	0.577	0.377	0.678	0.891	0.98
$L\text{-}C_SZW_{Rcavg}$	0.755	0.577	0.377	0.678	0.891	0.99
$L\text{-}C_SAW_{Rc}$	0.755	0.577	0.377	0.678	0.891	0.99
$L\text{-}C_SAW_{Rcavg}$	0.755	0.577	0.377	0.678	0.891	0.99

Table 19 List-wise results of both attention and zero weighting ($L\text{-}C_SZW$, $L\text{-}C_SAW$) with different reward methods (R_c and R_b) in both sum and average case ($_{\text{Avg}}$) and experience replays (R, PER) on LDos-Comoda Dataset with contextual variable "Mood"

Method	Precision@5	Precison@10	Precision@20	Recall@5	Recall@10	Recall@20
$L\text{-}C_SZW_{PERb}$	0.8888	0.6666	0.438	0.6934	0.8918	0.99
$L\text{-}C_SZW_{PERbavg}$	0.866	0.66666	0.438	0.6820	0.8918	0.99
$L\text{-}C_SAW_{PERb}$	0.84444	0.6555	0.41666	0.6744	0.8986	0.99
$L\text{-}C_SAW_{PERbavg}$	0.8666	0.66666	0.422222	0.67898	0.90161	0.99
$L\text{-}C_SZW_{PERc}$	0.7333	0.58888	0.383333	0.6466	0.8950	0.99
$L\text{-}C_SZW_{PERcavg}$	0.75555	0.6	0.3833	0.65956	0.8987	0.99
$L\text{-}C_SAW_{PERc}$	0.73333	0.5666	0.36111	0.5689	0.787	0.88888
$L\text{-}C_SAW_{PERcavg}$	0.7333	0.5666	0.366	0.5671	0.784	0.888
$L\text{-}C_SZW_{Rb}$	0.888	0.655	0.4333	0.689	0.8888	0.99
$L\text{-}C_SZW_{Rbavg}$	0.8888	0.655	0.437	0.691	0.892	0.989
$L\text{-}C_SAW_{Rb}$	0.8888	0.6666	0.4444	0.6920	0.888	0.992
$L\text{-}C_SAW_{Rbavg}$	**0.911**	**0.67777**	**0.444**	0.6934	0.8918	0.99
$L\text{-}C_SZW_{Rc}$	0.755	0.52222	0.35	0.61134	0.77113	0.99
$L\text{-}C_SZW_{Rcavg}$	0.7777	0.5333	0.361	0.615	0.767	0.99
$L\text{-}C_SAW_{Rc}$	0.8	0.6	0.3833	0.6897	0.8949	1.0
$L\text{-}C_SAW_{Rcavg}$	0.8	0.6	0.3833	0.6897	0.8949	0.99

Table 20 List-wise results of both attention and zero weighting (*L-C_SZW, L-C_SAW)* with different reward methods (R_c and R_b) in both sum and average case ($_{Avg}$) and experience replays (*R, PER*) on LDos-Comoda Dataset with contextual variable "day type"

With the contextual variable "location"						
Method	Precision@5	Precison@10	Precision@20	Recall@5	Recall@10	Recall@20
L-C_SZW$_{PERb}$	0.844	0.6444	0.4333	0.675	0.88	0.98
L-C_SZW$_{PERbavg}$	0.866	0.65	0.44	0.679	0.88	0.98
L-C_SAW$_{PERb}$	0.866	0.64	0.433	0.68	0.88	0.99
L-C_SAW$_{PERbavg}$	0.866	0.64	0.433	0.68	0.88	0.97
L-C_SZW$_{PERc}$	0.8	0.611	0.377	0.688	0.9120	0.98
L-C_SZW$_{PERcavg}$	0.8	0.611	0.377	0.688	0.9120	0.99
L-C_SAW$_{PERc}$	0.73	0.5777	0.3666	0.5799	0.7944	0.888
L-C_SAW$_{PERcavg}$	0.73	0.5777	0.3666	0.5799	0.7944	0.888
L-C_SZW$_{Rb}$	0.88	0.688	0.4388	0.6756	0.8979	0.97
L-C_SZW$_{Rbavg}$	**0.88**	**0.688**	**0.44**	0.674	0.894	0.98
L-C_SAW$_{Rb}$	**0.88**	**0.6888**	**0.444**	0.6688	0.899	0.98
L-C_SAW$_{Rbavg}$	**0.88**	**0.6888**	**0.444**	0.6688	0.899	0.97
L-C_SZW$_{Rc}$	0.666	0.5333	0.3500	0.4975	0.7883	0.888
L-C_SZW$_{Rcavg}$	0.64	0.51	0.33	0.49	0.78	0.88
L-C_SAW$_{Rc}$	0.755	0.5888	0.3722	0.6694	0.9027	0.91
L-C_SAW$_{Rcavg}$	0.733	0.577	0.3611	0.666	0.903	0.98

Table 21 List-wise results of both attention and zero weighting (*L-C_SZW, L-C_SAW)* with different reward methods (R_c and R_b) in both sum and average case ($_{Avg}$) and experience replays (*R, PER*) on LDos-Comoda Dataset with contextual variable "Location"

With the contextual variable "day type"						
Method	Precision@5	Precison@10	Precision@20	Recall@5	Recall@10	Recall@20
L-C_SZW$_{PERb}$	0.8888	0.655	0.427	0.7039	0.8921	0.97
L-C_SZW$_{PERbavg}$	0.888	0.644	0.4222	0.705	0.888	0.97
L-C_SAW$_{PERb}$	**0.9111**	**0.6888**	**0.444**	0.6817	0.8982	0.88
L-C_SAW$_{PERbavg}$	0.888	0.677	0.433	0.6788	0.8979	0.88
L-C_SZW$_{PERc}$	0.777	0.566	0.3611	0.6968	0.898	0.99
L-C_SZW$_{PERcavg}$	0.8	0.577	0.3777	0.7002	0.8917	0.98
L-C_SAW$_{PERc}$	0.711	0.555	0.355	0.58290	0.7825	0.888
L-C_SAW$_{PERcavg}$	0.711	0.5666	0.37	0.58	0.7875	0.888
L-C_SZW$_{Rb}$	0.888	0.666	0.4333	0.6826	0.8918	0.99
L-C_SZW$_{Rbavg}$	0.888	0.666	0.4333	0.6827	0.8918	0.97
L-C_SAW$_{Rb}$	0.888	0.677	0.4444	0.6687	0.8888	0.99
L-C_SAW$_{Rbavg}$	0.888	0.677	0.4444	0.6687	0.8888	0.98
L-C_SZW$_{Rc}$	0.7777	0.5666	0.3616	0.6927	0.88	0.88
L-C_SZW$_{Rcavg}$	0.777	0.5777	0.3777	0.6897	0.888	0.89
L-C_SAW$_{Rc}$	0.888	0.6777	0.4444	0.6687	0.8888	0.98
L-C_SAW$_{Rcavg}$	0.7111	0.5444	0.355	0.5316	0.784	0.888

Table 22 List-wise results of both attention and zero weighting ($L\text{-}C_SZW$, $L\text{-}C_SAW$) with different reward methods (R_c and R_b) in both sum and average case ($_{Avg}$) and experience replays (R, PER) on Depaul Movie dataset with contextual variable "Location"

With the contextual variable "Location"						
Method	**Precision@5**	**Precison@10**	**Precision@20**	**Recall@5**	**Recall@10**	**Recall@20**
$L\text{-}C_SZW_{PERb}$	**0.6631**	**0.394**	**0.20**	0.917	0.898	0.99
$L\text{-}C_SZW_{PERbavg}$	0.6631	0.39473	0.1973	0.9171	0.89	0.99
$L\text{-}C_SAW_{PERb}$	0.6105	0.368	0.1842	0.911	0.98	0.99
$L\text{-}C_SAW_{PERbavg}$	0.6315	0.3789	0.189473	0.9144	0.99	0.99
$L\text{-}C_SZW_{PERc}$	0.4105	0.252	0.12631	0.76754	0.8421	0.8421
$L\text{-}C_SZW_{PERcavg}$	0.4105	0.2526	0.1263	0.81641	0.894	0.89473
$L\text{-}C_SAW_{PERc}$	0.4	0.2473	0.123684	0.79962	0.89473	0.89473
$L\text{-}C_SAW_{PERcavg}$	0.3789	0.231	0.115789	0.77330	0.84210	0.8421
$L\text{-}C_SZW_{Rb}$	0.5894	0.36315	0.18157	0.8941	0.98	0.98
$L\text{-}C_SZW_{Rbavg}$	0.5894	0.3631	0.181578	0.8941	0.97	0.97
$L\text{-}C_SAW_{Rb}$	0.6	0.3684	0.1842	0.9117	0.99	0.99
$L\text{-}C_SAW_{Rbavg}$	0.6	0.3684	0.1842	0.9117	0.98	0.99
$L\text{-}C_SZW_{Rc}$	0.4631	0.2684210	0.13421	0.8320	0.89473	0.89473
$L\text{-}C_SZW_{Rcavg}$	0.4526	0.2631	0.1315	0.83208	0.8947	0.89473
$L\text{-}C_SAW_{Rc}$	0.4736	0.27368	0.13684	0.8381	0.894736	0.89473
$L\text{-}C_SAW_{Rcavg}$	0.4736	0.27894	0.1394	0.8250	0.89473	0.89473

Table 23 List-wise results of both attention and zero weighting ($L\text{-}C_SZW$, $L\text{-}C_SAW$) with different reward methods (R_c and R_b) in both sum and average case ($_{Avg}$) and experience replays (R, PER) on Depaul Movie Dataset with contextual variable "time"

With the contextual variable "time"						
Method	Precision@5	Precison@10	Precision@20	Recall@5	Recall@10	Recall@20
$L\text{-}C_SZW_{PERb}$	0.6210	0.3684	0.1842	0.91604	0.99	0.99
$L\text{-}C_SZW_{PERbavg}$	0.6315	0.3789	0.189473	0.9152	0.99	0.99
$L\text{-}C_SAW_{PERb}$	**0.6421**	**0.3894**	**0.1947**	0.898	0.99	1.0
$L\text{-}C_SAW_{PERbavg}$	**0.6421**	**0.3894**	**0.1947**	0.898	0.99	0.99
$L\text{-}C_SZW_{PERc}$	0.3789	0.242	0.12105	0.6692	0.7894	0.78947
$L\text{-}C_SZW_{PERcavg}$	0.3789	0.23684	0.11842	0.69561	0.78947	0.7894
$L\text{-}C_SAW_{PERc}$	0.4736	0.2631	0.13157	0.8557	0.89473	0.894
$L\text{-}C_SAW_{PERcavg}$	0.4736	0.263	0.1315	0.8557	0.8947	0.8947
$L\text{-}C_SZW_{Rb}$	0.6210	0.3684	0.18421	0.9217	0.99	0.99
$L\text{-}C_SZW_{Rbavg}$	0.6210	0.3684	0.18421	0.92178	0.99	0.99
$L\text{-}C_SAW_{Rb}$	0.6210	0.3736	0.1868	0.9010	0.99	1.0
$L\text{-}C_SAW_{Rbavg}$	0.6210	0.373684	0.18684210	0.90100	0.99	0.99
$L\text{-}C_SZW_{Rc}$	0.442	0.2631	0.13157	0.7666	0.84210	0.84210
$L\text{-}C_SZW_{Rcavg}$	0.4	0.2421	0.1210	0.76403	0.8421	0.8421
$L\text{-}C_SAW_{Rc}$	0.4	0.242105	0.1210	0.65726	0.7368	0.73684
$L\text{-}C_SAW_{Rcavg}$	0.378	0.231578	0.11578947	0.604	0.6842	0.6842

Table 24 List-wise results of both attention and zero weighting (*L-C_SZW, L-C_SAW)* with different reward methods (R_c and R_b) in both sum and average case ($_{Avg}$) and experience replays (*R, PER*) on Depaul Movie dataset with contextual variable "companion"

With the contextual variable "companion"						
Method	Precision@5	Precison@10	Precision@20	Recall@5	Recall@10	Recall@20
L-C_SZW$_{PERb}$	**0.6526**	**0.399**	**0.199**	0.902	0.99	0.99
L-C_SZW$_{PERbavg}$	**0.6526**	**0.399**	**0.19999**	0.9050	0.99	0.99
L-C_SAW$_{PERb}$	0.6315	0.3578	0.1789	0.94152	0.99	0.99
L-C_SAW$_{PERbavg}$	0.621	0.3526	0.1763	0.9385	0.99	0.99
L-C_SZW$_{PERc}$	0.4105	0.2578	0.12894	0.6988	0.7894	0.78947
L-C_SZW$_{PERcavg}$	0.4421	0.27368	0.13684	0.7061	0.78947	0.7894
L-C_SAW$_{PERc}$	0.4631	0.25263	0.12631	0.8135	0.842105	0.842
L-C_SAW$_{PERcavg}$	0.4631	0.2578	0.128	0.803	0.8421	0.8421
L-C_SZW$_{Rb}$	0.652	0.3789	0.1894	0.9356	0.99	1.0
L-C_SZW$_{Rbavg}$	0.652	0.3789	0.1894	0.93567	0.99	0.99
L-C_SAW$_{Rb}$	0.6105	0.37368	0.18684	0.9070	0.99	0.99
L-C_SAW$_{Rbavg}$	0.6210	0.37894	0.1894	0.90705	0.99	1.0
L-C_SZW$_{Rc}$	0.410	0.231	0.1157	0.8484	0.8947	0.8947
L-C_SZW$_{Rcavg}$	0.4315	0.2473	0.12368	0.8381	0.894	0.8947
L-C_SAW$_{Rc}$	0.4210	0.2473	0.12368	0.726	0.7894	0.7894
L-C_SAW$_{Rcavg}$	0.4210	0.2473	0.12368	0.726	0.7894	0.7894

Appendix I

See Tables 16, 17, 18, 19, 20, 21, 22, 23, 24

Author contribution M.B and M.M finalized the problem formulation; F.A., and M.B, completed the formal analysis; M.B., and M.M., carried out the implementation; M.B, and M.M, finalized the Experiments and Results; F.A. and M.M finalized Figures and Typesetting; F.A., and M.M, wrote the paper; M.B. and M.M. reviewed and finalized the draft.

Declarations

Competing Interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

Adomavicius G, Tuzhilin A (2005) Toward the next generation of recommender systems: a survey of the state-of-the-art and possible extensions. IEEE Trans Knowl Data Eng 17(6):734–749

Adomavicius G, Tuzhilin A (2011) Context-aware recommender systems. Recommender systems handbook. Springer, Cham, pp 217–253
Bi K et al. (2019) Conversational product search based on negative feedback. In: Proceedings of the 28th ACM International Conference on Information and Knowledge Management
Casillo M et al (2022) Context aware recommender systems: a novel approach based on matrix factorization and contextual bias. Electronics 11(7):1003
Chen L, Xia M (2021) A context-aware recommendation approach based on feature selection. Appl Intell 51(2):865–875
Chen X et al (2021) A survey of deep reinforcement learning in recommender systems: A systematic review and future directions. arXiv preprint arXiv:2109.03540
Cui L et al (2018) A novel context-aware recommendation algorithm with two-level SVD in social networks. Futur Gener Comput Syst 86:1459–1470
D'Amico E et al (2022) Analyzing and improving stability of matrix factorization for recommender systems. J Intel Inform Syst 58(2):255–285
Dhelim S et al (2022) A hybrid personality-aware recommendation system based on personality traits and types models. J Ambient Intel Humanized Comput. https://doi.org/10.1007/s12652-022-04200-5
Dhruv A et al (2019) Artist recommendation system using hybrid method: a novel approach. Emerging research in computing, information, communication and applications. Springer, Cham, pp 527–542
Ding Z, Dong H (2020) Challenges of reinforcement learning. Deep reinforcement learning: fundamentals, research and applications. Springer, Singapore, pp 249–272
Dong M et al (2022) A survey for trust-aware recommender systems: a deep learning perspective. Knowl-Based Syst 249:108954
Frolov E and Oseledets I (2016) Fifty shades of ratings: how to benefit from a negative feedback in top-N recommendations tasks. In: Proceedings of the 10th ACM Conference on Recommender Systems.
Ghasemi N, Momtazi S (2021) Neural text similarity of user reviews for improving collaborative filtering recommender systems. Electron Commer Res Appl 45:101019
Guo L et al (2019) Collaborative filtering recommendation based on trust and emotion. J Intel Inform Syst 53:113–135
Hu B, Shi C and Liu J (2017) Playlist recommendation based on reinforcement learning. In: International Conference on Intelligence Science. Springer, Cham
Huang L et al (2021) A deep reinforcement learning based long-term recommender system. Knowl-Based Syst 213:106706
Iqbal M et al (2019) Kernel context recommender system (KCR): a scalable context-aware recommender system algorithm. IEEE Access 7:24719–24737
Isinkaye FO, Folajimi YO, Ojokoh BA (2015) Recommendation systems: principles, methods and evaluation. Egyptian Inform J 16(3):261–273
Jaques N et al. (2016) Tuning recurrent neural networks with re-inforcement learning. arXiv preprint arXiv:1611.02796
Jeong S-Y, Kim Y-K (2021) Deep learning-based context-aware recommender system considering contextual features. Appl Sci 12(1):45
Jin J et al. (2018) Real-time bidding with multi-agent reinforcement learning in display advertising. In: Proceedings of the 27th ACM International Conference on Information and Knowledge Management
Kumar C, Chowdary CR, Shukla D (2022) Automatically detecting groups using locality-sensitive hashing in group recommendations. Inf Sci 601:207–223
Lam KY, Lee LH, and Hui P (2021) A2w: Context-aware recommendation system for mobile augmented reality web browser. In: Proceedings of the 29th ACM International Conference on Multimedia
Lample G and Chaplot DS (2017) Playing FPS games with deep reinforcement learning. In Thirty-First AAAI Conference on Artificial Intelligence
Lei Y, Li W (2019) Interactive recommendation with user-specific deep reinforcement learning. ACM Tran Knowl Discov Data (TKDD) 13(6):1–15
Li X et al (2018) A multi-dimensional context-aware recommendation approach based on improved random forest algorithm. IEEE Access 6:45071–45085
Li X et al (2021) A context-aware diversity-oriented knowledge recommendation approach for smart engineering solution design. Knowl-Based Syst 215:106739
Lin Y et al (2023) A survey on reinforcement learning for recommender systems. IEEE Trans Neural Netw Learn Syst. https://doi.org/10.1109/TNNLS.2023.3280161
Liu F et al (2020) State representation modeling for deep reinforcement learning based recommendation. Knowl-Based Syst 205:106170
Liu F et al. (2018) Deep reinforcement learning based recommendation with explicit user-item interactions modeling. arXiv preprint arXiv:1810.12027

Liu Z, Wen S and Quan Y (2021) Deep Reinforcement Learning based Group Recommender System. arXiv preprint arXiv:2106.06900
Liu Y et al. (2021) Diversity-promoting deep reinforcement learning for interactive recommendation. In: 5th International Conference on Crowd Science and Engineering
Lu J, Wu D, Mao M, Wang W, Zhang G (2015) Recommender system application developments: a survey. Decis Support Syst 74:12–32
Luong M-T, Pham H and Manning CD (2015) Effective approaches to attention-based neural machine translation. arXiv preprint arXiv:1508.04025
Mnih V et al. (2013) Playing Atari with deep reinforcement learning. arXiv preprint arXiv:1312.5602
Musto C et al (2021) Context-aware graph-based recommendations exploiting Personalized PageRank. Knowl-Based Syst 216:106806
Neary P (2018) *Automatic hyperparameter tuning in deep convolutional neural networks using asynchronous reinforcement learning*. In: 2018 IEEE International Conference on Cognitive Computing (ICCC). IEEE.
Nichols D, Oki B, Terry D (1992) Using collaborative filtering to weave an information tapestry. Comm Assoc Comput Mach 35:51–60
Oliverio J (2018) A survey of social media, big data, data mining, and analytics. J Ind Integr Manag 3(03):1850003
Pan Y et al. (2022) Understanding and mitigating the limitations of prioritized experience replay. In: Uncertainty in Artificial Intelligence. PMLR.
Pérez-Almaguer Y et al (2021) Content-based group recommender systems: a general taxonomy and further improvements. Expert Syst Appl 184:115444
Rahim A et al. (2020) A Trust Assisted Matrix Factorization based Improved Product Recommender System. In: 2020 International Conference on Computational Science and Computational Intelligence (CSCI). IEEE.
Raza S, Ding C (2019) Progress in context-aware recommender systems—An overview. Comput Sci Rev 31:84–97
Rostami M, Oussalah M, Farrahi V (2022) A novel time-aware food recommender-system based on deep learning and graph clustering. IEEE Access 10:52508–52524
Sattar A, Bacciu D (2022) Graph neural network for context-aware recommendation. Neural Proc Lett. https://doi.org/10.1007/s11063-022-10917-3
Sattar A, Bacciu D (2023) Graph neural network for context-aware recommendation. Neural Process Lett 55(5):5357–5376
Schröder G, Thiele M, and Lehner W (2011) Setting goals and choosing metrics for recommender system evaluations. In: UCERSTI2 workshop at the 5th ACM conference on recommender systems, Chicago, USA
Shani G et al (2005) An MDP-based recommender system. J Mach Learn Res 6(9):1265–1295
Sharma RS, Shaikh AA, Li E (2021) Designing recommendation or suggestion systems: looking to the future. Electron Mark 31(2):243–252
Tai C-Y et al. (2021) User-centric path reasoning towards explainable recommendation. In: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval
Taneja A, Arora A (2019) Modeling user preferences using neural networks and tensor factorization model. Int J Inf Manag 45:132–148
Unger M, Tuzhilin A (2020) Hierarchical latent context representation for context-aware recommendations. IEEE Trans Knowl Data Eng 34(7):3322–3334
Unger M, Tuzhilin A, Livne A (2020) Context-aware recommendations based on deep learning frameworks. ACM Trans Manag Inform Syst (TMIS) 11(2):1–15
Vaidya, N. and A. Khachane. *Recommender systems-the need of the ecommerce ERA*. in *2017 International Conference on Computing Methodologies and Communication (ICCMC)*. 2017. IEEE.
Wu D et al. (2018) Budget constrained bidding by model-free reinforcement learning in display advertising. In: Proceedings of the 27th ACM International Conference on Information and Knowledge Management
Xin X et al. (2020) Self-supervised reinforcement learning for recommender systems. In: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in INFORMATION Retrieval
Yousefian Jazi S, Kaedi M, Fatemi A (2021) An emotion-aware music recommender system: bridging the user's interaction and music recommendation. Multimed Tools Appl 80(9):13559–13574
Yu Y (2018) Towards sample efficient reinforcement learning. in IJCAI.
Yuyan Z, Xiayao S and Yong L (2019) A novel movie recommendation system based on deep reinforcement learning with prioritized experience replay. In: 2019 IEEE 19th International Conference on Communication Technology (ICCT). IEEE.

Zhao X, Zhang W and Wang J (2013) Interactive collaborative filtering. In: Proceedings of the 22nd ACM international conference on Information & Knowledge Management
Zhao X et al. (2018) Recommendations with negative feedback via pairwise deep reinforcement learning. In: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining
Zhao X et al. (2018) Deep reinforcement learning for page-wise recommendations. In: Proceedings of the 12th ACM Conference on Recommender Systems
Zhao Z et al (2021) Tag-aware recommender system based on deep reinforcement learning. Mathem. Probl Eng 2021:5564234
Zhao X et al. (2021) Dear: Deep reinforcement learning for online advertising impression in recommender systems. In: Proceedings of the AAAI Conference on Artificial Intelligence
Zheng Y (2022a) Context-aware collaborative filtering using context similarity: an empirical comparison. Information 13(1):42
Zheng Y (2022b) DeepCARSKit: A deep learning based context-aware recommendation library. Software Impacts 13:100292
Zheng G et al. (2018) DRN: A deep reinforcement learning framework for news recommendation. In: Proceedings of the 2018 World Wide Web Conference
Zikopoulos P, Eaton C (2011) Understanding big data: analytics for enterprise class hadoop and streaming data. McGraw-Hill Osborne Media, New York
Zimdars A, Chickering DM and Meek C (2013) Using temporal data for making recommendations. arXiv preprint arXiv:1301.2320

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.